[ADS5034-01] Computer Vision

07. Features Matching

Jae-Young Choi

Sungkyunkwan University

성균관대학교 SUNGKYUNKWAN UNIVERSITY

Outline



- Feature Matching
  - Fitting, RANSAC
- Image Alignment
  - Homography
  - Rectification
  - Stitching

What is next?

- Local features: main components

Find feature!

- **Detection:** Identify the interest points

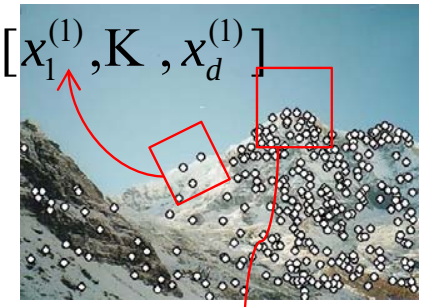
Represent feature!

- **Description:** Extract vector feature descriptor surrounding each interest point.

Match feature!

- **Matching:** Determine correspondence between descriptors in two views







Fitting, Alignment, RANSAC

Fitting

- Condition
  - **input is a set of points**
  - **Noise** in the measured feature locations
  - **Extraneous data:** clutter (outliers), multiple lines
  - **Missing data:** occlusions
- If we know which points belong to the line, how do we find the “optimal” line parameters? *Least squares*
- What if there are outliers? *Robust fitting, RANSAC*
- What if there are many lines? *Voting methods: RANSAC, Hough transform*
- What if we’re not even sure it’s a line? *Model selection*

Fitting

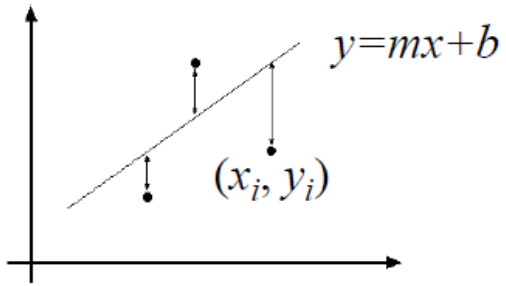
- Least squares line fitting

Data: $(x_1, y_1), \ldots, (x_n, y_n)$

Line equation: $y_i = m x_i + b$

Find (m, b) to minimize

$$E = \sum_{i=1}^{n} (y_i - m x_i - b)^2$$



Problem with "vertical" least squares

- Not rotation-invariant
- Fails completely for vertical lines

$$Y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \qquad X = \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \qquad B = \begin{bmatrix} m \\ b \end{bmatrix}$$

$$E = \|Y - XB\|^2 = (Y - XB)^T (Y - XB) = Y^T Y - 2(XB)^T Y + (XB)^T (XB)$$

$$\frac{dE}{dB} = 2X^T XB - 2X^T Y = 0$$

$$X^T XB = X^T Y$$

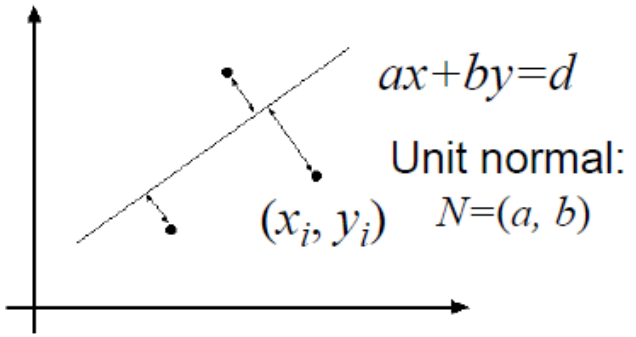
Normal equations: least squares solution to $XB = Y$

Fitting

- Total least squares

Distance between point (x_i, y_i) and line $ax+by=d$ $(a^2+b^2=1)$: $|ax_i + by_i - d|$

Find (a, b, d) to minimize the sum of squared perpendicular distances



$$E = \sum_{i=1}^{n} (ax_i + by_i - d)^2$$

$$\frac{\partial E}{\partial d} = \sum_{i=1}^{n} -2(ax_i + by_i - d) = 0$$

$$d = \frac{a}{n}\sum_{i=1}^{n} x_i + \frac{b}{n}\sum_{i=1}^{n} y_i = a\bar{x} + b\bar{y}$$

$$E = \sum_{i=1}^{n} (a(x_i - \bar{x}) + b(y_i - \bar{y}))^2 = \left\| \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} \right\|^2 = (UN)^T (UN)$$

$$\frac{dE}{dN} = 2(U^T U)N = 0$$

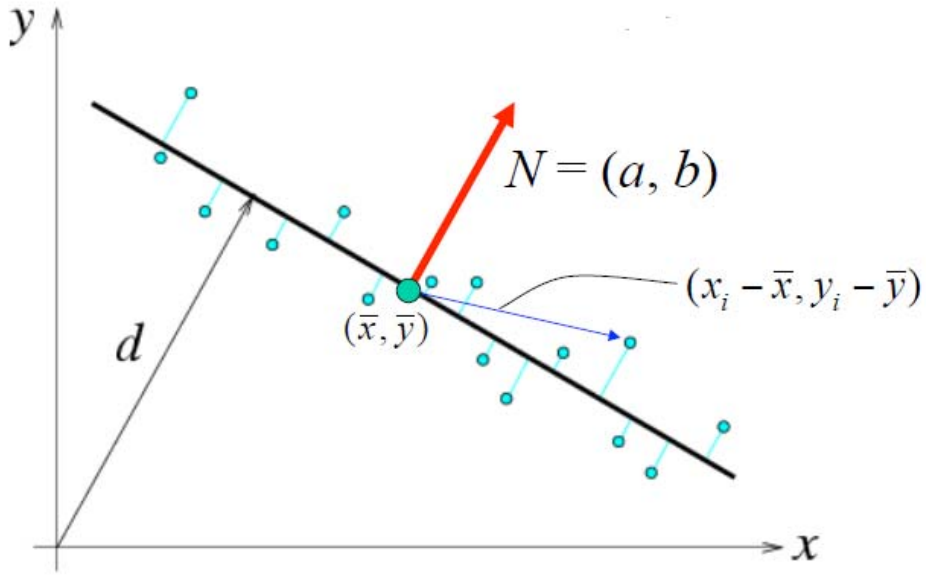
Fitting

- Total least squares
  - Solution to $(U^T U)N = 0$, subject to $\left|\left|N\right|\right|^2 = 1$: eigenvector of $U^T U$ associated with the smallest eigenvalue (least squares solution to homogeneous linear system $UN = 0$)

$$U = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix}$$

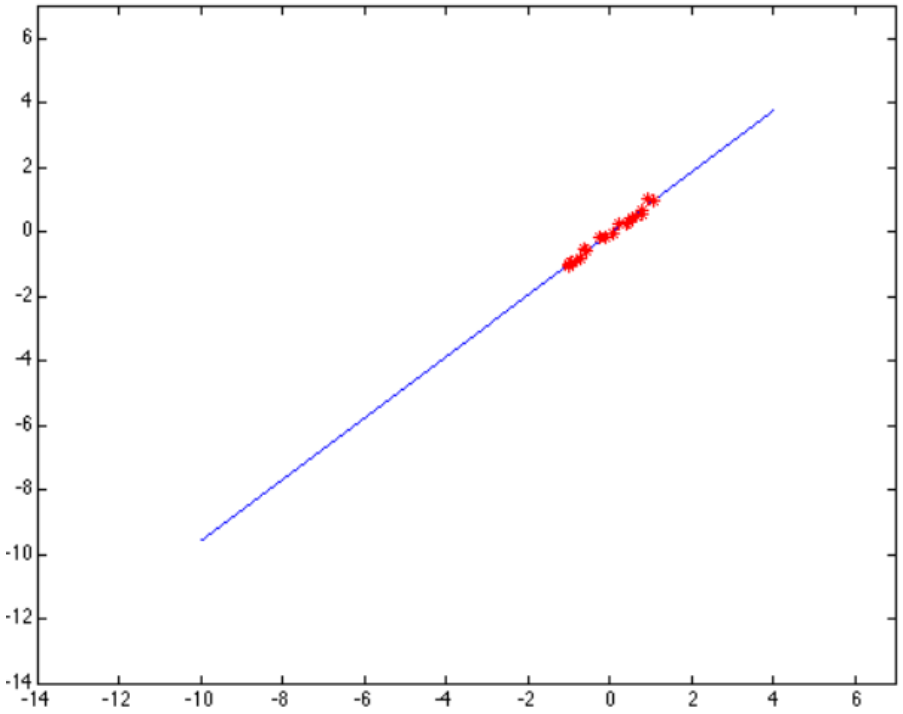
$$U^T U = \begin{bmatrix} \sum_{i=1}^{n} (x_i - \bar{x})^2 & \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) \\ \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) & \sum_{i=1}^{n} (y_i - \bar{y})^2 \end{bmatrix}$$

second moment matrix



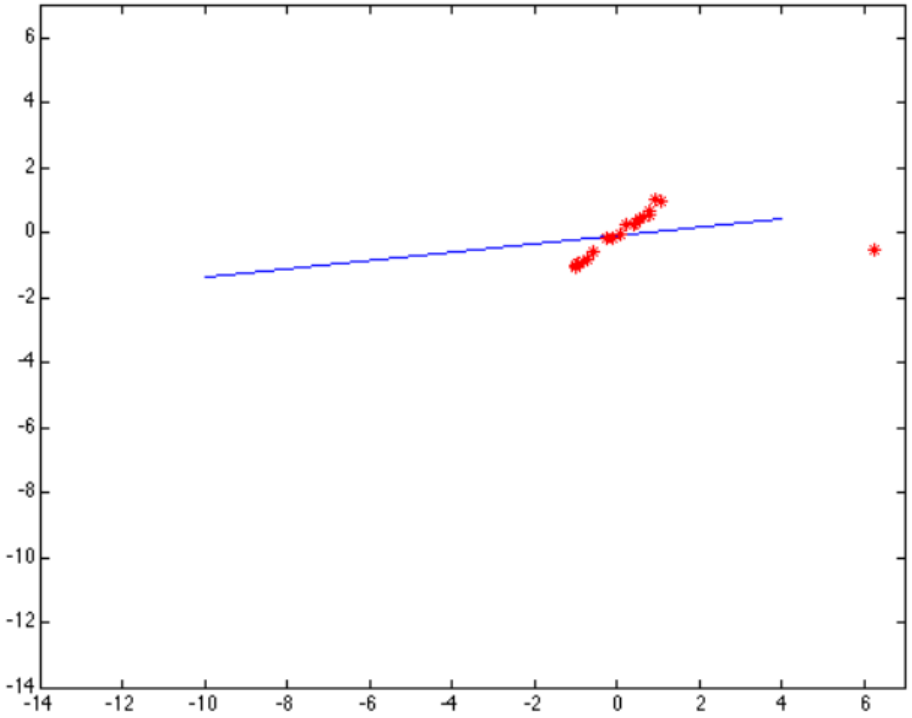
Fitting

- Least squares: Robustness to noise
  - Least squares fit to the red points:



Fitting

- Least squares: Robustness to noise
  - Least squares fit with an outlier:



Problem:
squared error heavily penalizes outliers

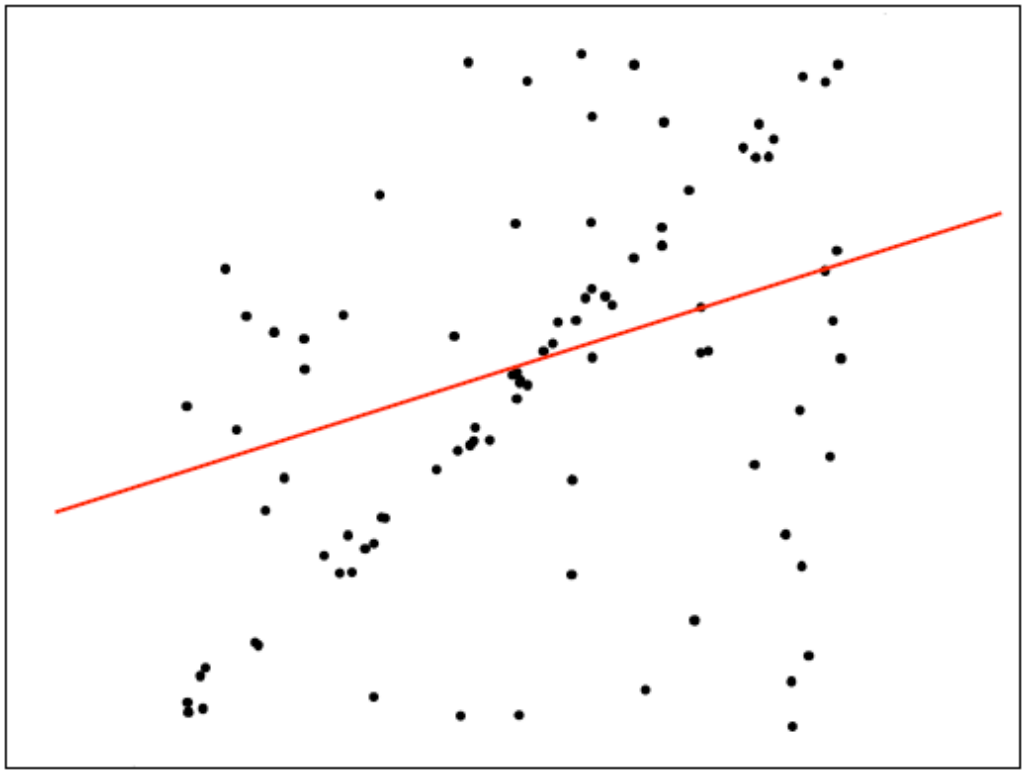
RANSAC

- Robust fitting can deal with a few outliers
- RANdom SAmple Consensus (RANSAC):
  - Very general framework for model fitting in the presence of outlier
- Outline
  - Choose a small subset of points uniformly at random
  - Fit a model to that subset
  - Find all remaining points that are “close” to the model and reject the rest as outliers
  - Do this many times and choose the best model

M. A. Fischler, R. C. Bolles.
Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Comm. of the ACM, Vol 24, pp381-395, 1981.

RANSAC

- Line fitting example

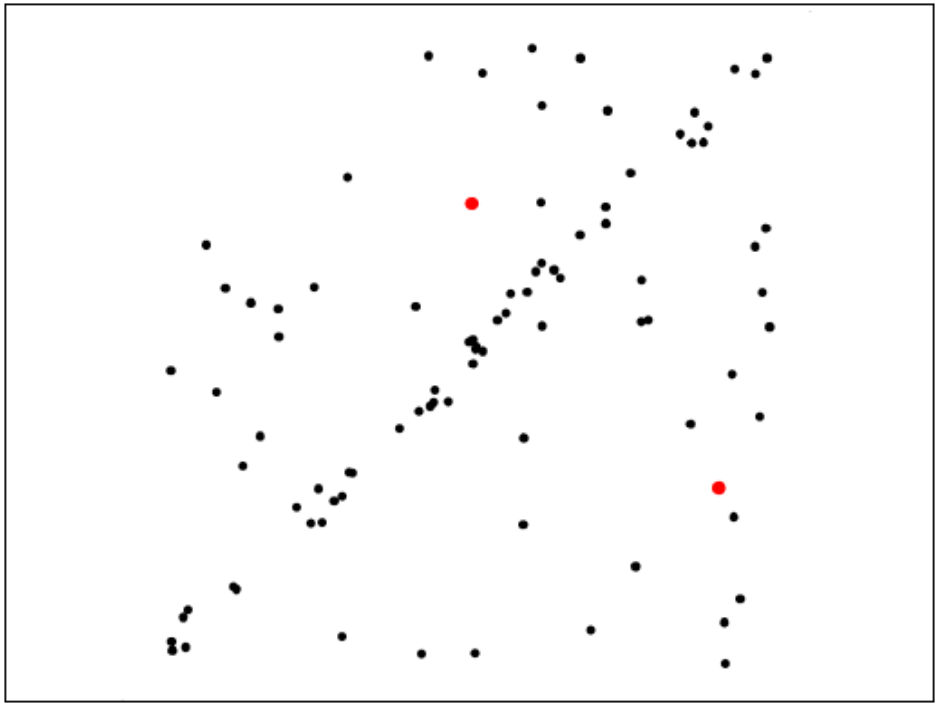


Least-squares fit

RANSAC

- Line fitting example

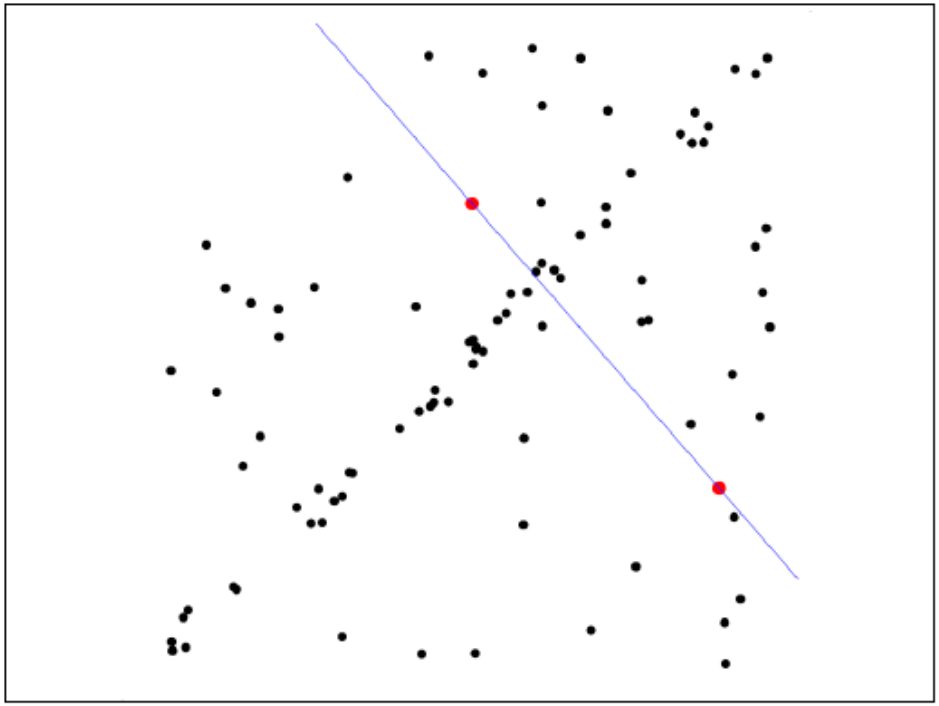
1. Randomly select minimal subset of points



RANSAC

- Line fitting example

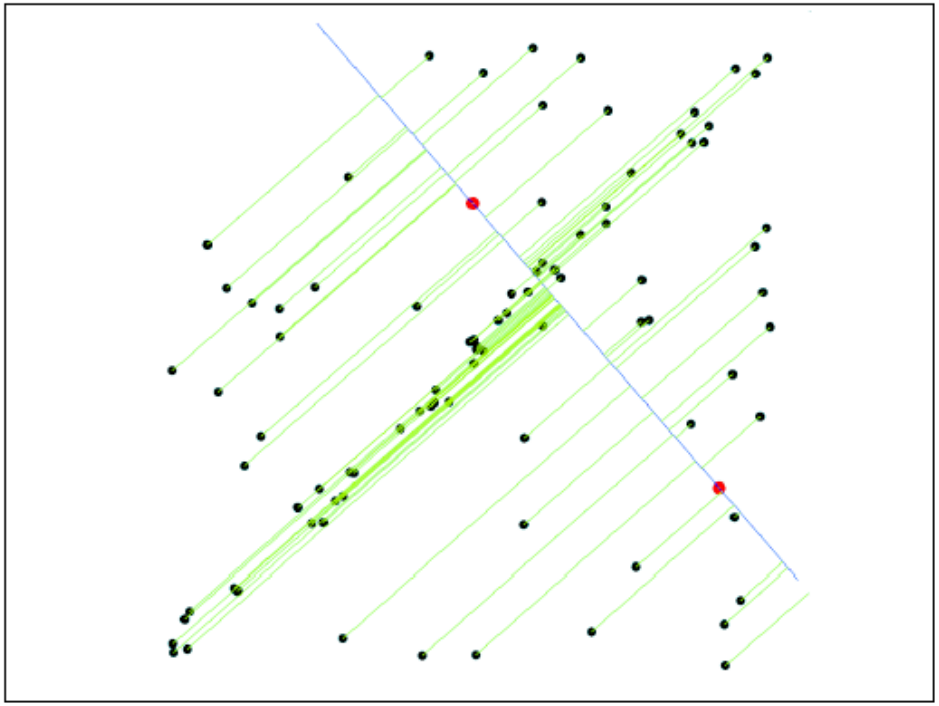
1. Randomly select minimal subset of points
2. Hypothesize a model



RANSAC

- Line fitting example

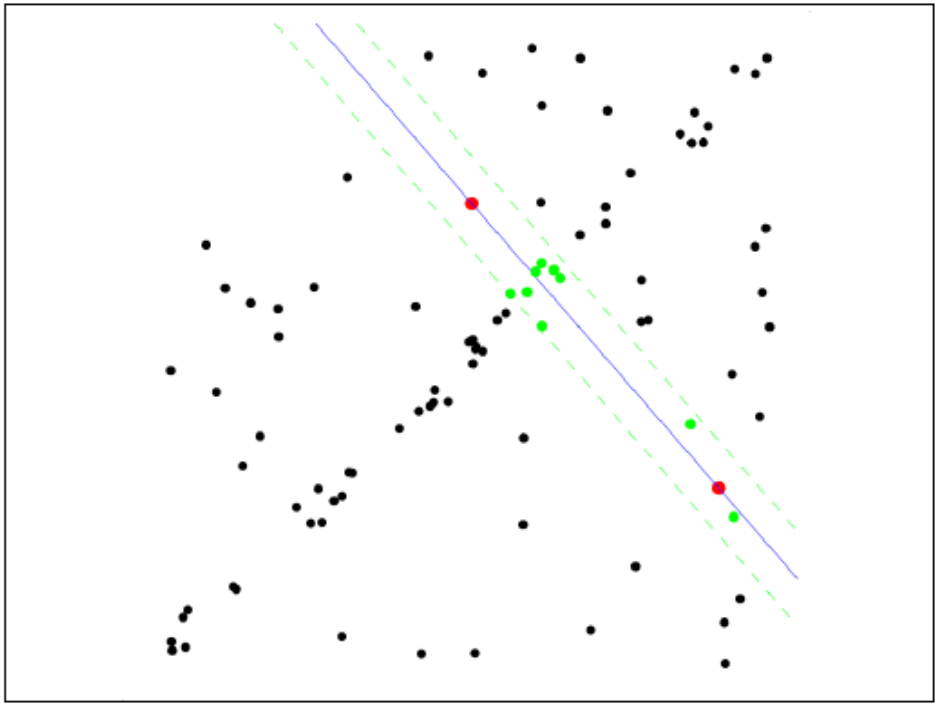
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function



RANSAC

- Line fitting example

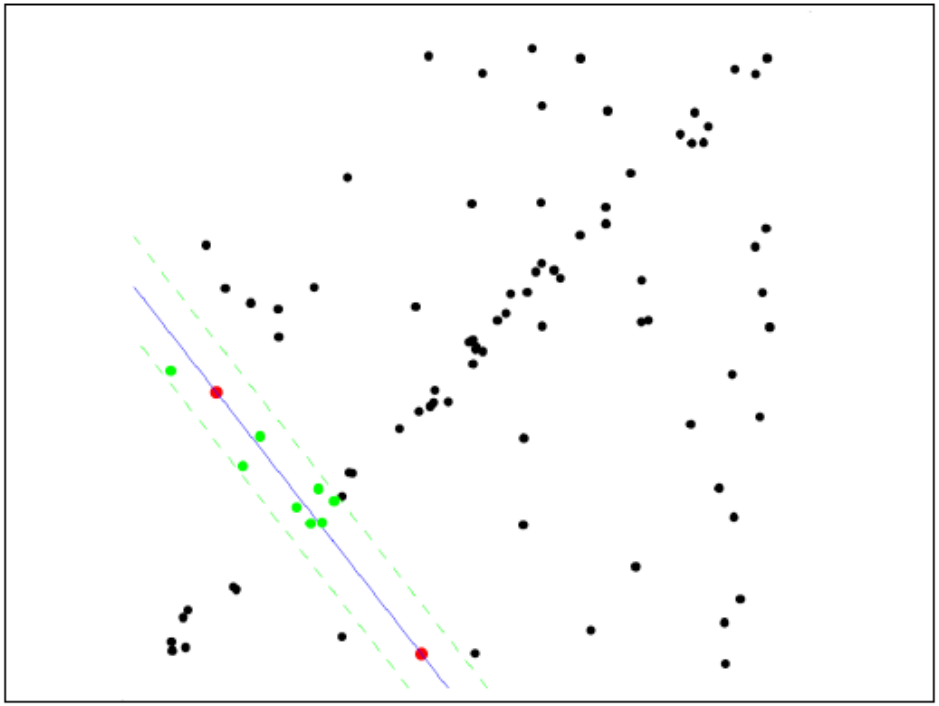
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model



RANSAC

- Line fitting example

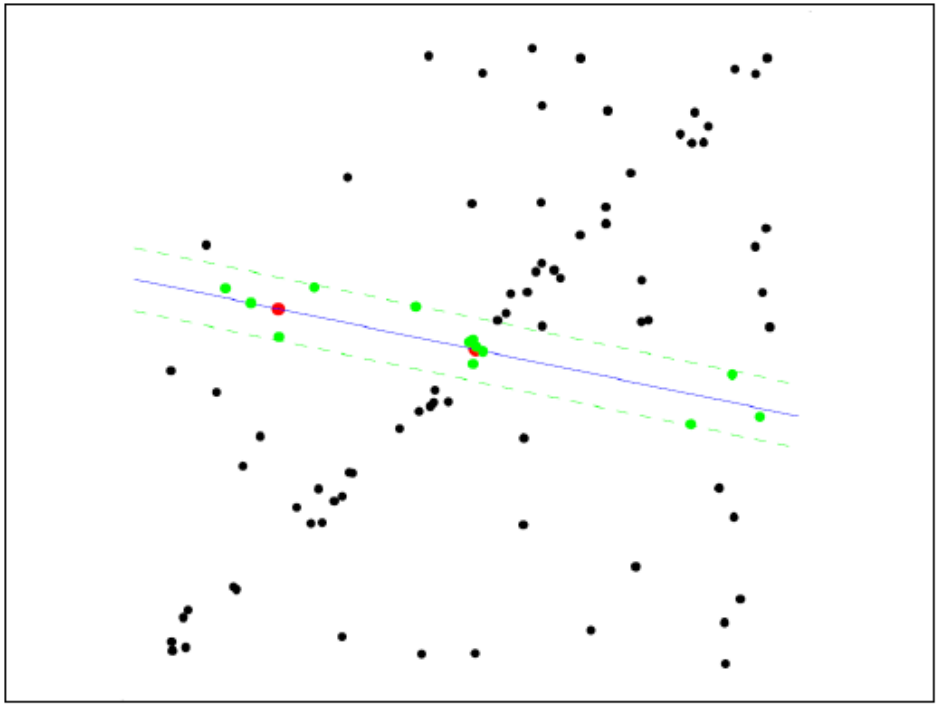
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat hypothesize and verify loop



RANSAC

- Line fitting example

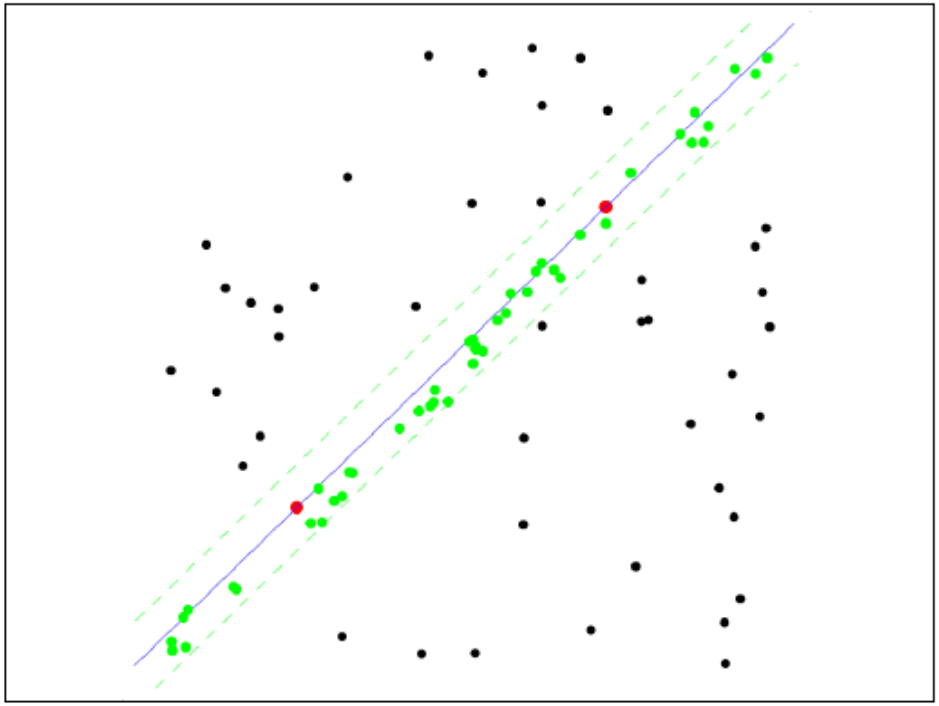
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat hypothesize and verify loop



RANSAC

- Line fitting example

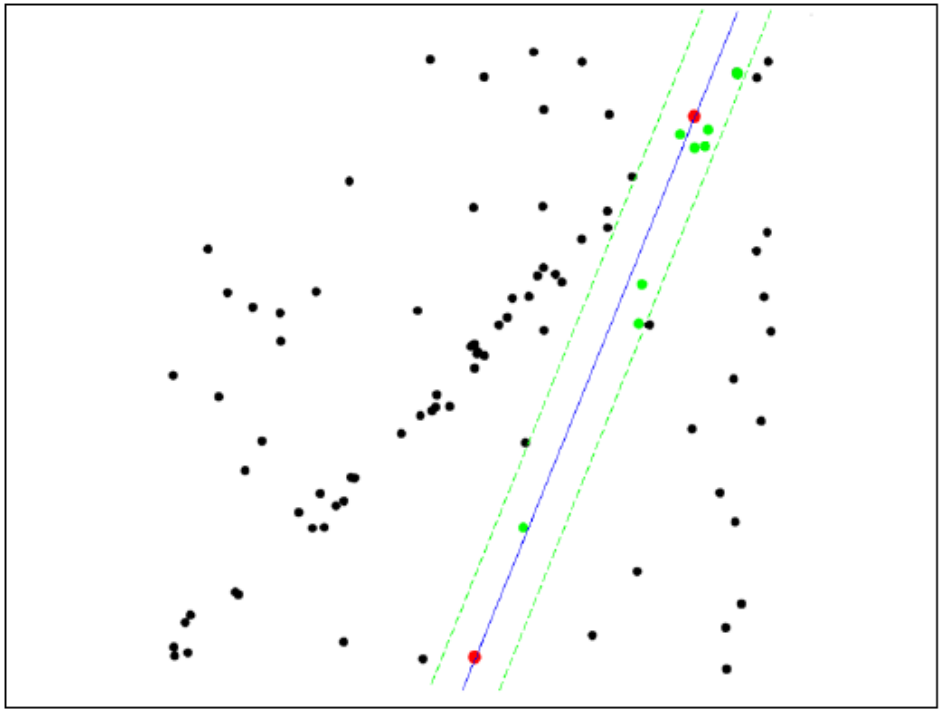
1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat hypothesize and verify loop



RANSAC

- Line fitting example

1. Randomly select minimal subset of points
2. Hypothesize a model
3. Compute error function
4. Select points consistent with model
5. Repeat hypothesize and verify loop



RANSAC

- Line fitting example
- Repeat ***N*** times:
  - Draw ***s*** points uniformly at random
  - Fit line to these ***s*** points
  - Find inliers to this line among the remaining points (i.e., points whose distance from the line is less than ***t***)
  - If there are ***d*** or more inliers, accept the line and refit using all inliers

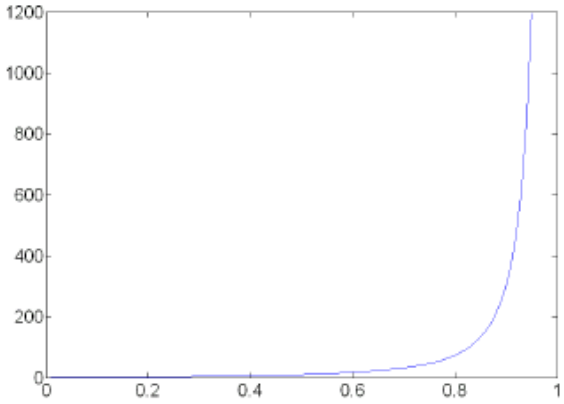
RANSAC

<Choosing the parameters>

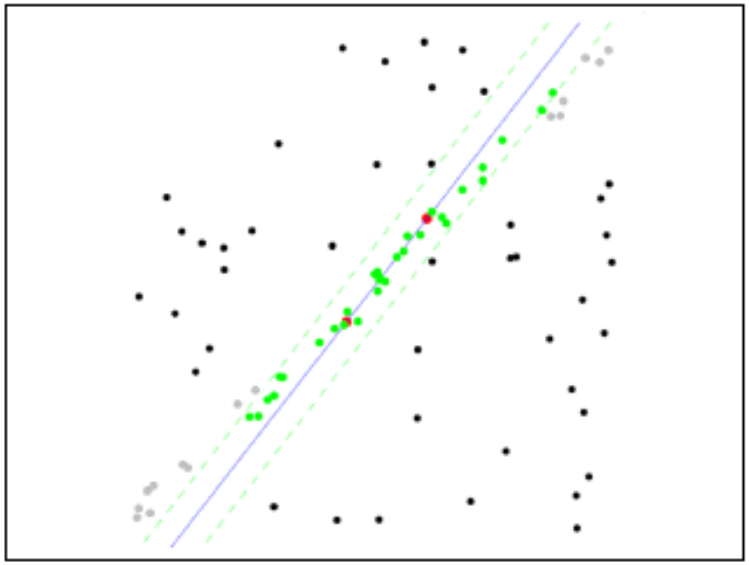
- Initial number of points *s*
  - Typically minimum number needed to fit the model
- Distance threshold *t*
  - Choose *t* so probability for inlier is *p* (e.g. 0.95)
  - Zero-mean Gaussian noise with std. dev. σ: $t^2=3.84\sigma^2$
- Number of samples *N*
  - Choose *N* so that, with probability *p*, at least one random sample is free from outliers (e.g. *p*=0.99) (outlier ratio: *e*)
- Consensus set size *d*
  - Should match expected inlier ratio

$$\left(1-\left(1-e\right)^{s}\right)^{N}=1-p$$

$$N=\log\left(1-p\right)/\log\left(1-\left(1-e\right)^{s}\right)$$



RANSAC



- Pros
  - Simple and general
  - Applicable to many different problems
  - Often works well in practice
- Cons
  - Lots of parameters to tune
  - Doesn’t work well for low inlier ratios (too many iterations, or can fail completely)
  - Can’t always get a good initialization of the model based on the minimum number of samples

Another Feature Matching

https://docs.opencv.org/master/dc/dc3/tutorial_py_matcher.html

- Brute-Force Matcher
  1. Take the descriptor of one feature in first set
  2. Match with all other features in second set
  3. Calculate distance using some distance calculation
  4. The closest one is returned
- FLANN Matcher
  - FLANN : Fast Library for Approximate Nearest Neighbors
  - It contains a collection of algorithms optimized for fast nearest neighbor search in large datasets and for high dimensional features
  - It works more faster than Brute-Force Matcher for large datasets

Correspondence

- So far, we have assumed that we are given a set of ground-truth correspondences between two images
- What if we do not know the correspondence?



Correspondence

1. Extract features (detection and description)
2. Compute *potential matches*
3. Loop: *(1) Hypothesize* transformation T

   (2) Verify transformation (search for other matches consistent with T)



Image Alignment

- A look into the past

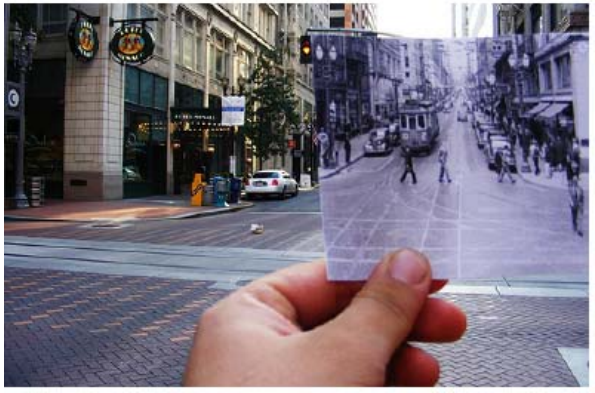


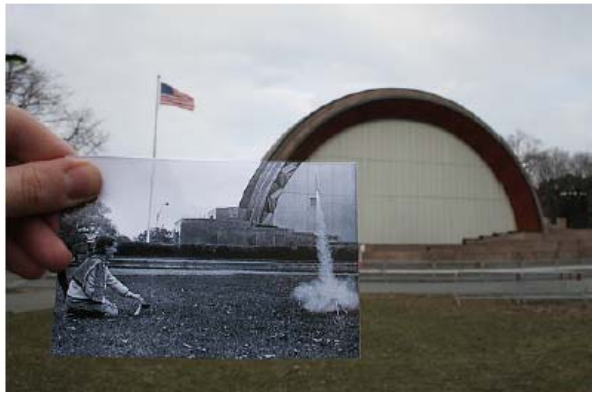


http://komen-dant.livejournal.com/345684.html
Leningrad during the blockade









http://blog.flickr.net/en/2010/01/27/a-look-into-the-past/

Image Alignment

- Challenges

- Small degree of overlap
- Occlusion, Clutter
- Intensity changes





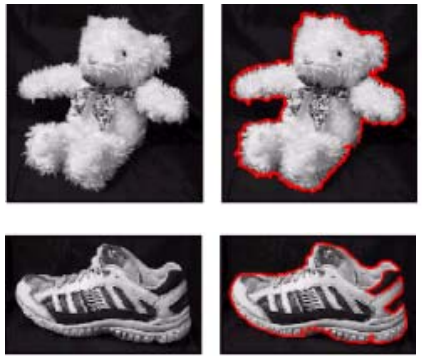
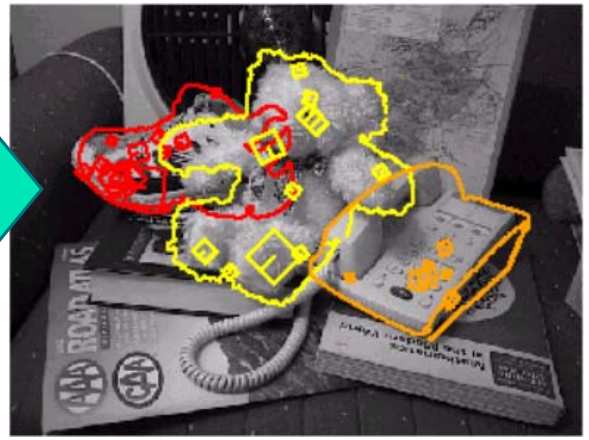
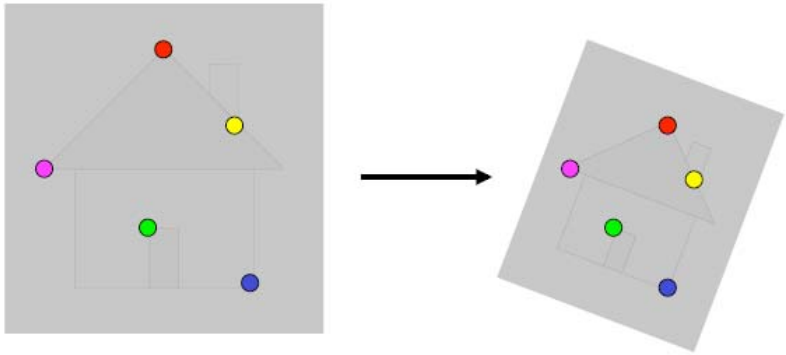



Image Alignment

- Two families of approaches:

- **Direct (pixel-based) alignment**
  - Search for alignment where most pixels agree



- **Feature-based alignment**
  - Search for alignment where extracted features agree
  - Can be verified using pixel-based alignment

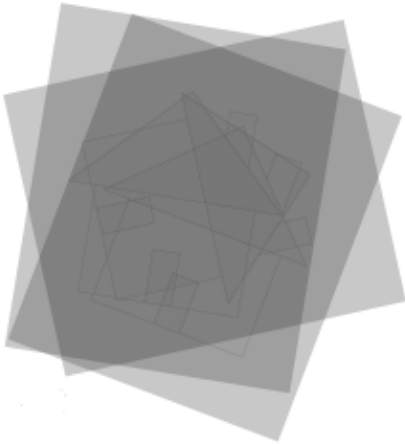
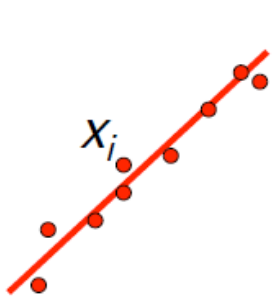


Image Alignment

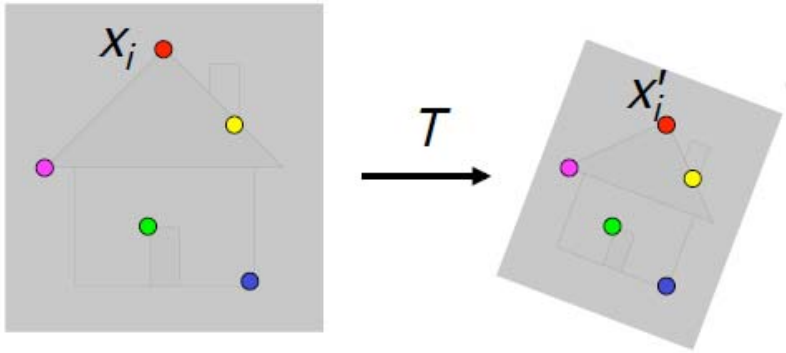
- Alignment as fitting:
- Previous slides:
  - **fitting** a model to features in one image



Find model M that minimizes

$$\sum_i \text{residual}(x_i, M)$$

- **Alignment**:
  - fitting a model to a transformation between pairs of features (matches) in two images



Find **transformation T** that minimizes

$$\sum_i \text{residual}(T(x_i), x_i')$$

Image Alignment

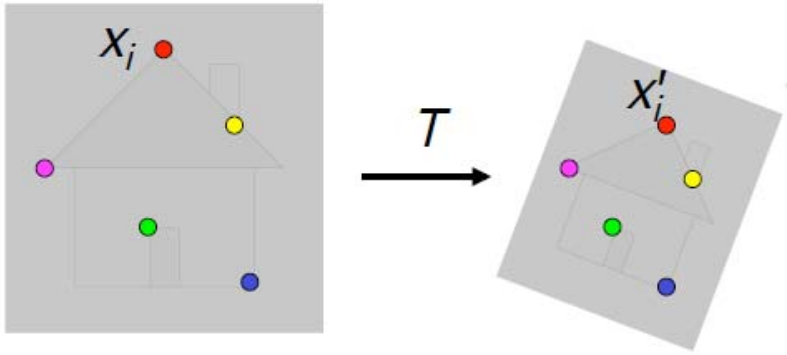
- Think about fitting an affine transformation:
  - Simple fitting procedure (linear least squares)
  - Approximates viewpoint changes for roughly planar objects and roughly orthographic cameras
  - Can be used to initialize fitting for more complex models





Image Alignment

- Think about fitting an affine transformation:
  - Assume we know the correspondences, how do we get the transformation?



$$\begin{bmatrix} x_i' \\ y_i' \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \longrightarrow \begin{bmatrix} & \cdots & & & & \\ x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ & \cdots & & & & \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \cdots \\ x_i' \\ y_i' \\ \cdots \end{bmatrix}$$

Image Alignment

- Think about fitting an affine transformation:

$$\begin{bmatrix} & & \cdots & & & \\ x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ & & \cdots & & & \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \cdots \\ x_i' \\ y_i' \\ \cdots \end{bmatrix}$$

  - Linear system with six unknowns
  - Each match gives us two linearly independent equations

    : need at least three to solve for the transformation parameters

Image Alignment

- Translate one image relative to another:










Translation-only
alignment is not enough!!

Image Alignment

- Translate one image relative to another:







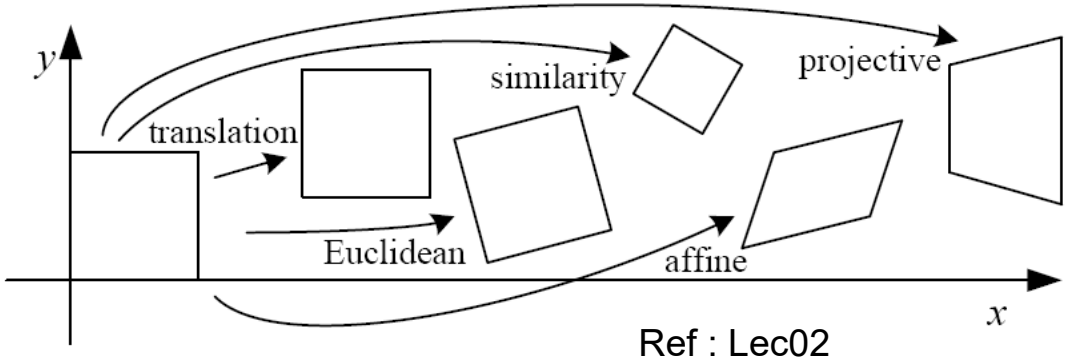



Use image homography !!

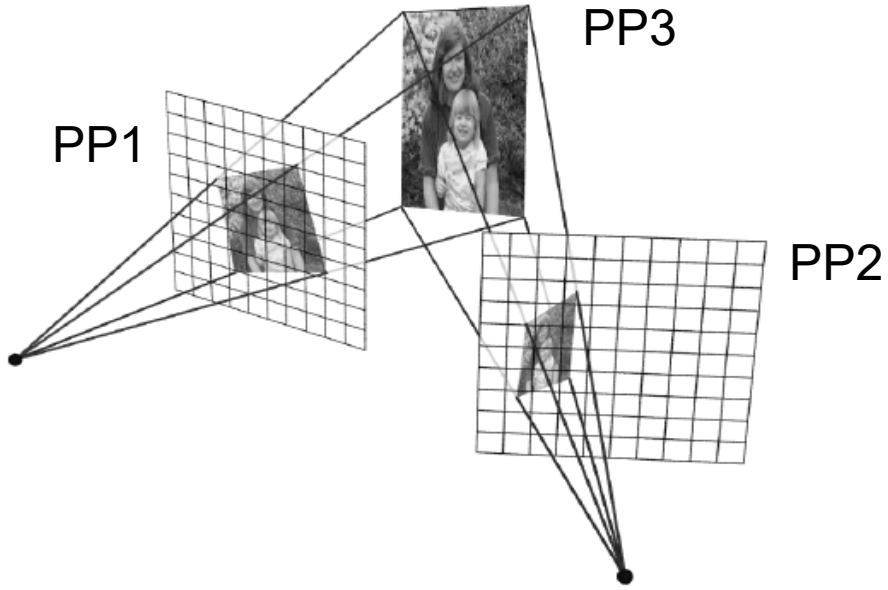
Homography
Rectification, Stitching

Homography

- PP3 is a projection plane of both centers of projection
- Which kind transformation is needed to warp projective plane 1(PP1) into projective plane 2(PP2)?



Ref : Lec02

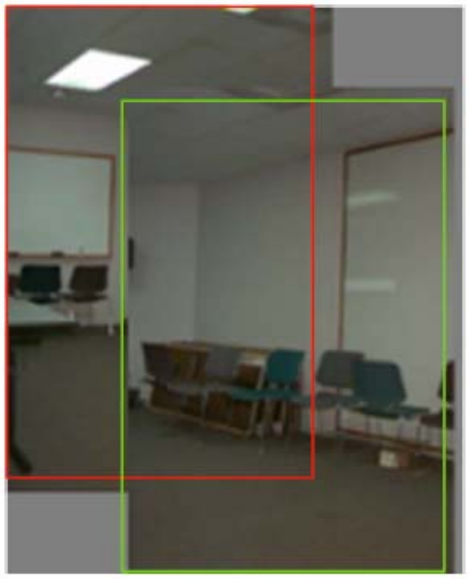


https://docs.opencv.org/master/da/d6e/tutorial_py_geometric_transformations.html

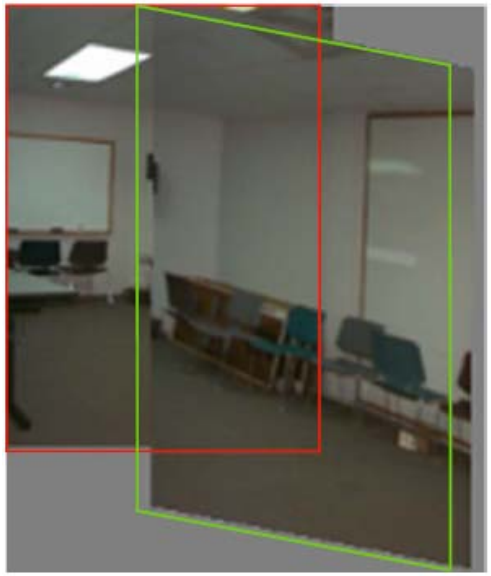
Homography

- Homograpy :
  - plane projective transformation

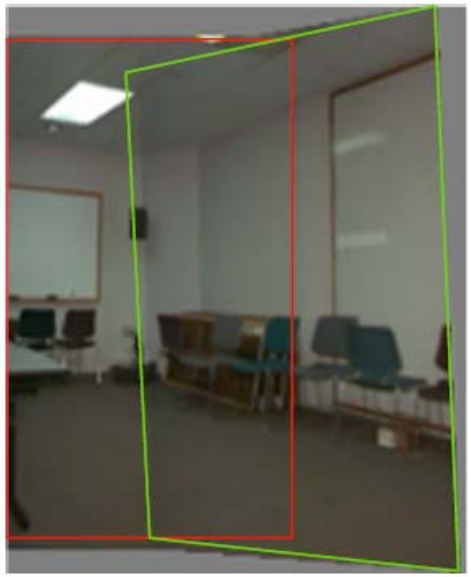
translation



affine



homography



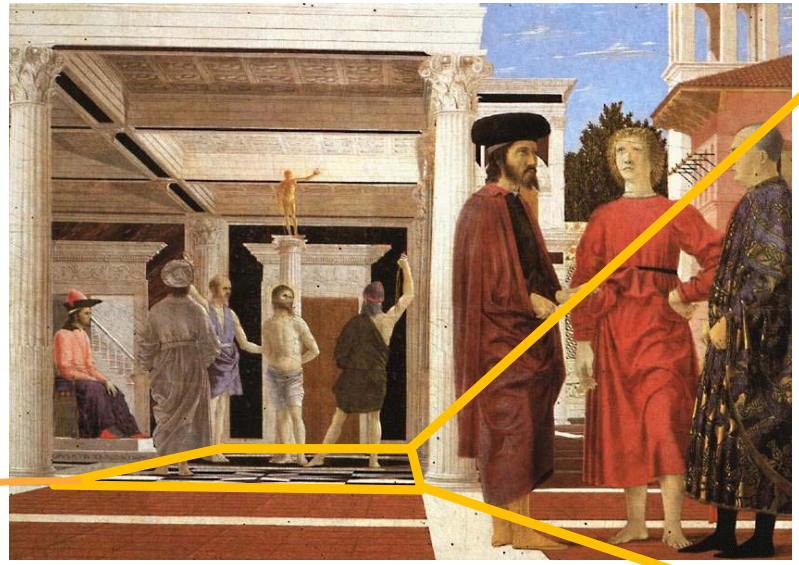
Homography

- Warping with transformations :
  - Street Arts



Homography

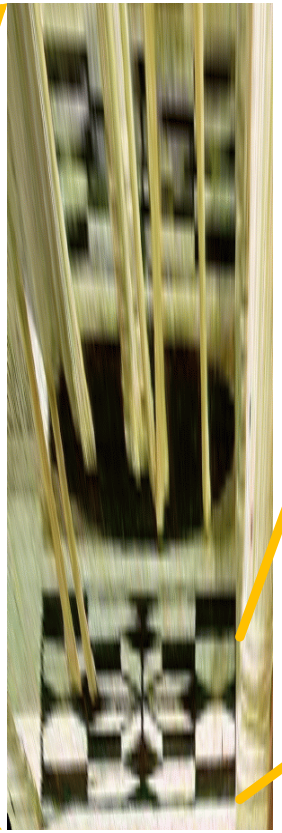
- Warping with transformations :
  - Understanding geometric patterns



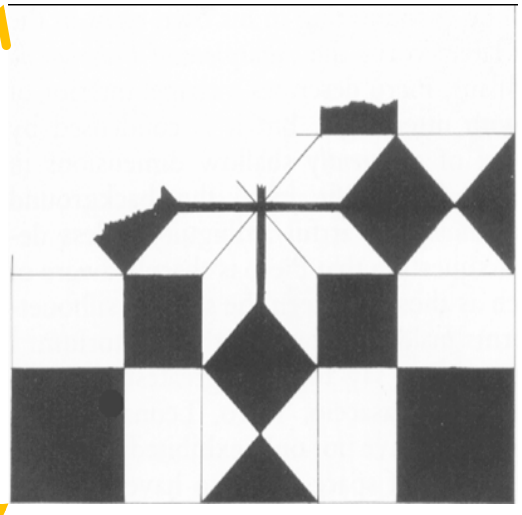


magnified view of floor

Homography



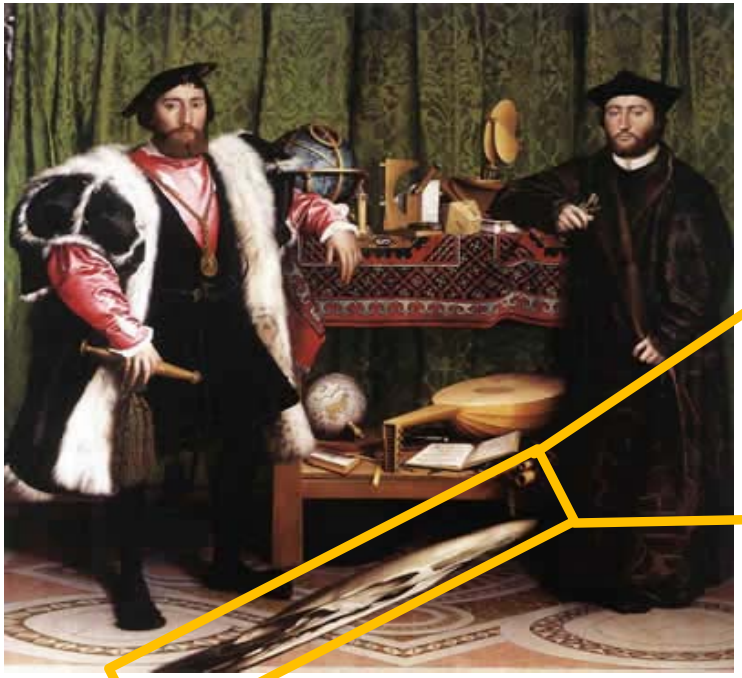
rectified view



reconstruction from rectified view

Homography

- Warping with transformations :
  - A weird drawing



Holbein, “The Ambassadors”

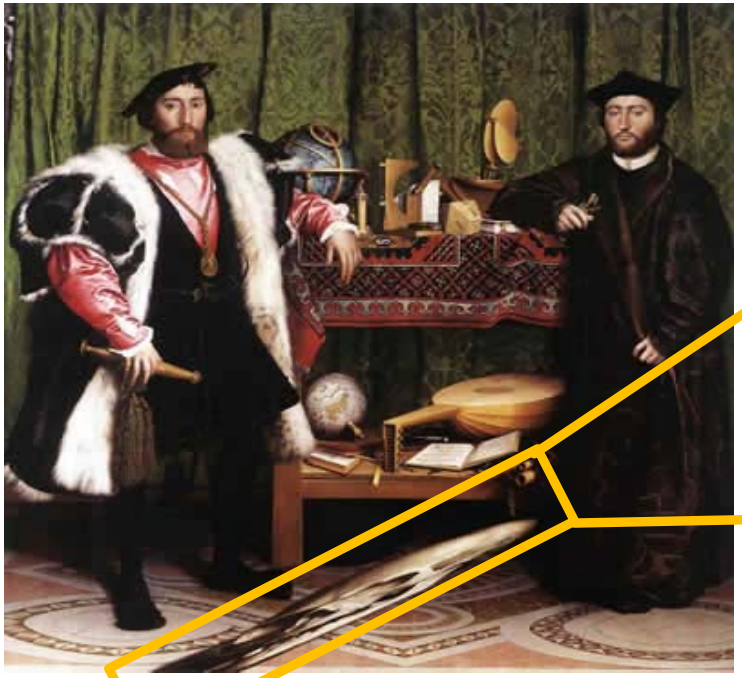


rectified view

(skull under anamorphic perspective)

Homography

- Warping with transformations :
  - A weird drawing



Holbein, “The Ambassadors”



DIY: use a polished spoon to see the skull

Homography

- Warping with transformations :
  - Virtual camera rotations



St.Petersburg photo by A.Tikhonov

original view





Synthetic rotations

Homography

- Warping with transformations :
  - Image rectification





two original images



rectified and stitched

Homography

- When Homography can be used :

  1. The transformation between two views of a planar surface

     (The scene is planar)







Homography

- When Homography can be used :

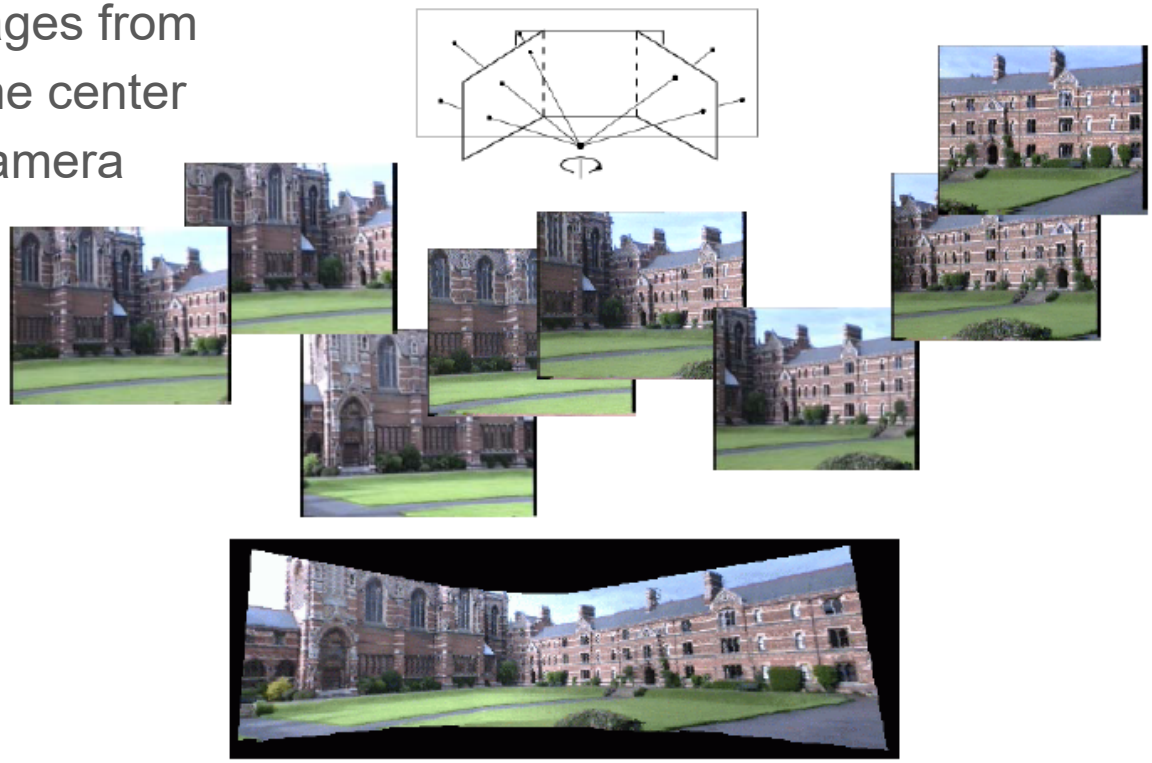
  2. The scene is very far or has small (relative) depth variation

     (Scene is approximately planar)



Homography

- When Homography can be used :
  3. The transformation between images from two cameras that share the same center (The scene is captured under camera rotation only)



Source: Hartley & Zisserman

Determining the Homography

1. Convert to homogeneous coordinates:

$$p = \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2. Multiply by the homography matrix :

$$P' = H \cdot P$$

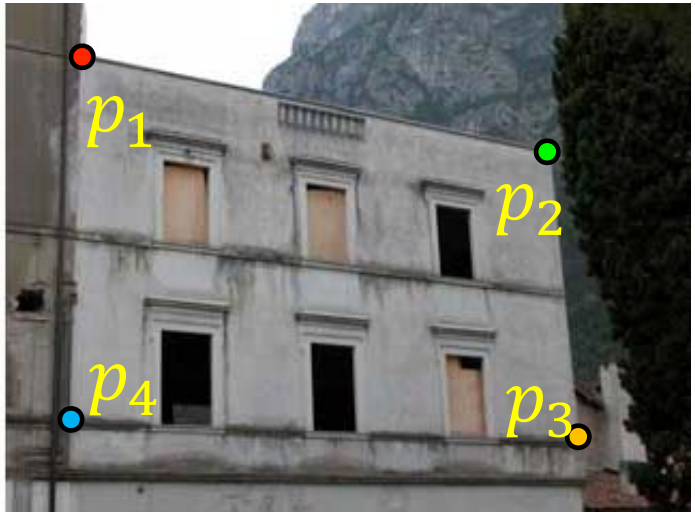
3. Convert back to heterogeneous coordinates:

$$P' = \begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} \Rightarrow p' = \begin{bmatrix} x'/w' \\ y'/w' \end{bmatrix}$$

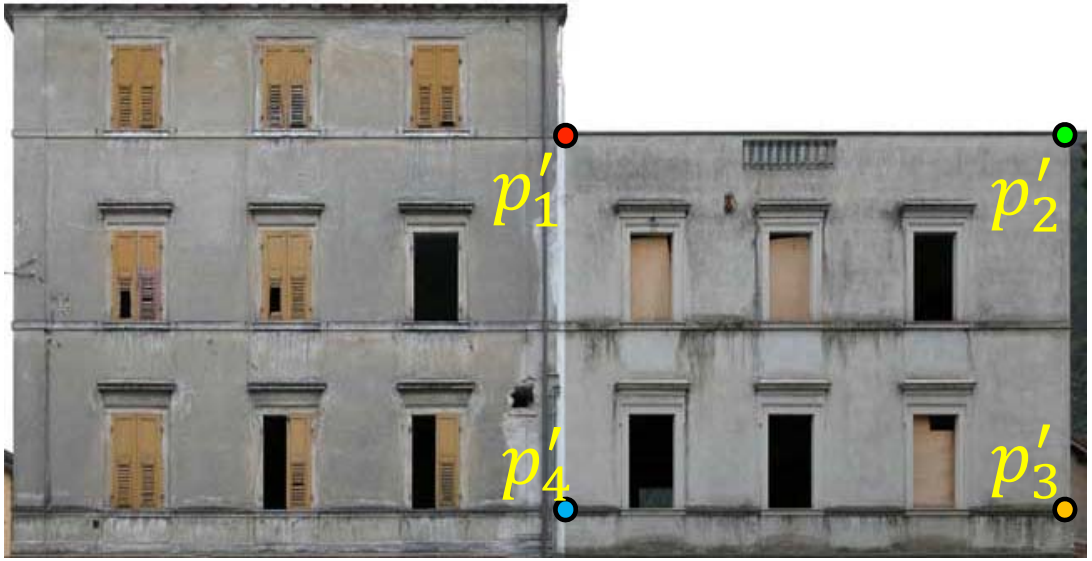
Determining the Homography

- Create point correspondences:
  - Given a set of matched feature points $\{p_i, p_i'\}$, find the best estimate of H such that

$$P' = H \cdot P$$



original image



target image

Determining the Homography

- Write out linear equation for each correspondence:

$$P' = H \cdot P$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{aligned} x' &= \alpha(h_1 x + h_2 y + h_3) \\ y' &= \alpha(h_4 x + h_5 y + h_6) \\ 1 &= \alpha(h_7 x + h_8 y + h_9) \end{aligned}$$

Expand the matrix multiplication

$$\begin{aligned} x'(h_7 x + h_8 y + h_9) &= (h_1 x + h_2 y + h_3) \\ y'(h_7 x + h_8 y + h_9) &= (h_4 x + h_5 y + h_6) \end{aligned}$$

Divide out unknown scale factor

Determining the Homography

- How do you rearrange terms to make it a linear system?

$$x'(h_7x + h_8y + h_9) = (h_1x + h_2y + h_3)$$
$$y'(h_7x + h_8y + h_9) = (h_4x + h_5y + h_6)$$

Rearrange the terms

$$h_7xx' + h_8yx' + h_9x' - h_1x - h_2y - h_3 = 0$$
$$h_7xy' + h_8yy' + h_9y' - h_4x - h_5y - h_6 = 0$$

Re-write in matrix form

$$\mathbf{A}_i\boldsymbol{h} = \mathbf{0}$$

$$\mathbf{A}_i = \begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\ 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{bmatrix}$$

$$\boldsymbol{h} = \begin{bmatrix} h_1 & h_2 & h_3 & h_4 & h_5 & h_6 & h_7 & h_8 & h_9 \end{bmatrix}^\top$$

Determining the Homography

- Stack together constraints from multiple point correspondences:

$$\mathbf{A}\boldsymbol{h} = \mathbf{0}$$

$$\begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\ 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{bmatrix}$$

$$\begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\ 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{bmatrix}$$

$$\vdots$$

$$\begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \\ 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \end{bmatrix}$$

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \\ h_9 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Determining the Homography

- Singular Value Decomposition (SVD)
  - Solution is the column of $\boldsymbol{V}$ corresponding to smallest singular value

diagonal

ortho-normal ortho-normal

$$\underset{n \times m}{\mathbf{A}} = \underset{n \times n}{\mathbf{U}} \underset{n \times m}{\mathbf{\Sigma}} \underset{m \times m}{\mathbf{V}^\top}$$

unit norm constraint

$$= \sum_{i=1}^{9} \sigma_i \underset{n \times 1}{\boldsymbol{u}_i} \underset{1 \times m}{\boldsymbol{v}_i^\top}$$

Determining the Homography

- Solving for H :
  - Given $\{x_i, x_i'\}$, solve for H such that $x' = Hx$

1. For each correspondence, create 2×9 matrix A_i
2. Concatenate into single $2n \times 9$ matrix A
3. Compute SVD of $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$
4. Store singular vector of the smallest singular value $h = v_{\hat{i}}$
5. Reshape to get H

Homography

- Image correspondence pipeline :

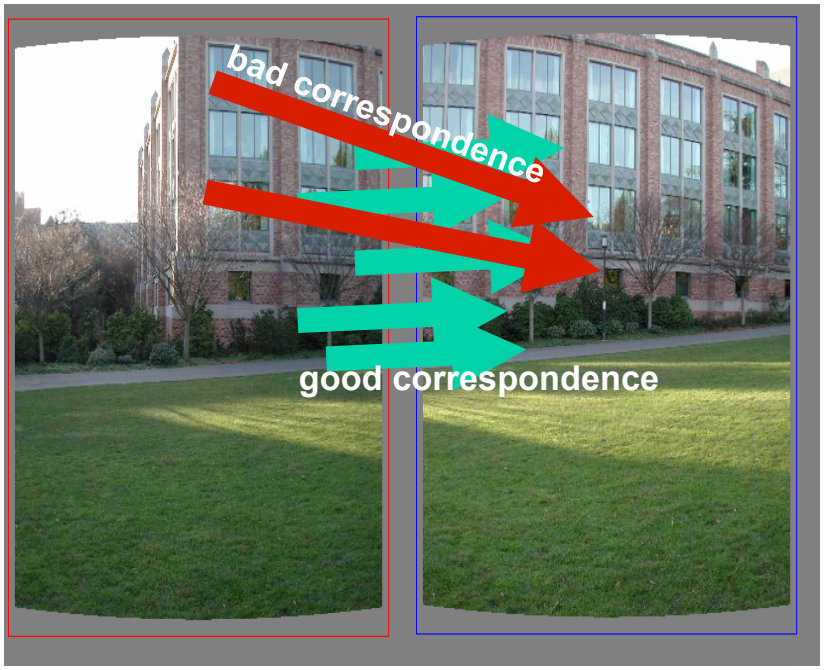
1. Feature point detection
   - Detect corners (e.g. using the Harris corner detector)

2. Feature point description
   - Describe features (e.g. using the Multi-scale oriented patch descriptor)

3. Feature matching and homography estimation
   - Do both simultaneously using RANSAC

Homography

- Matching Problem :
  - Linear least squares estimation only works when the transform function is linear!
  - Also doesn't deal well with **outliers.**
  - Using RANSAC



Homography

- Estimating homography using RANSAC :
  - Given two images…
  - find matching features (e.g., SIFT)

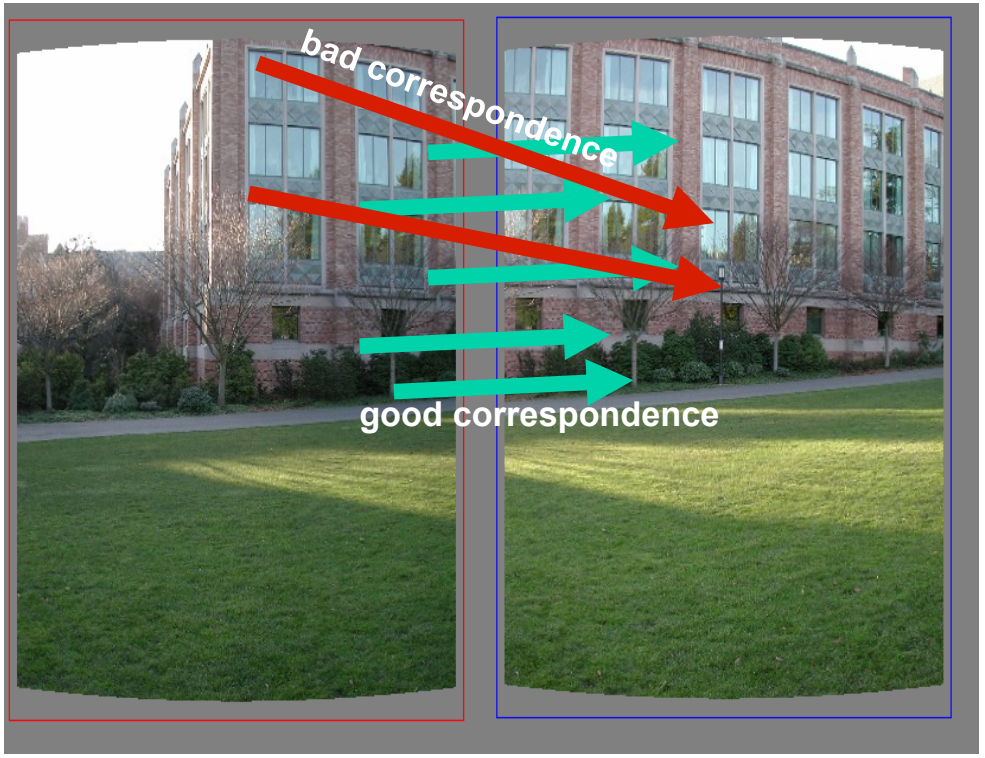
    and a translation transform



Homography

- Estimating homography using RANSAC :
  - Matched points will usually contain bad correspondences

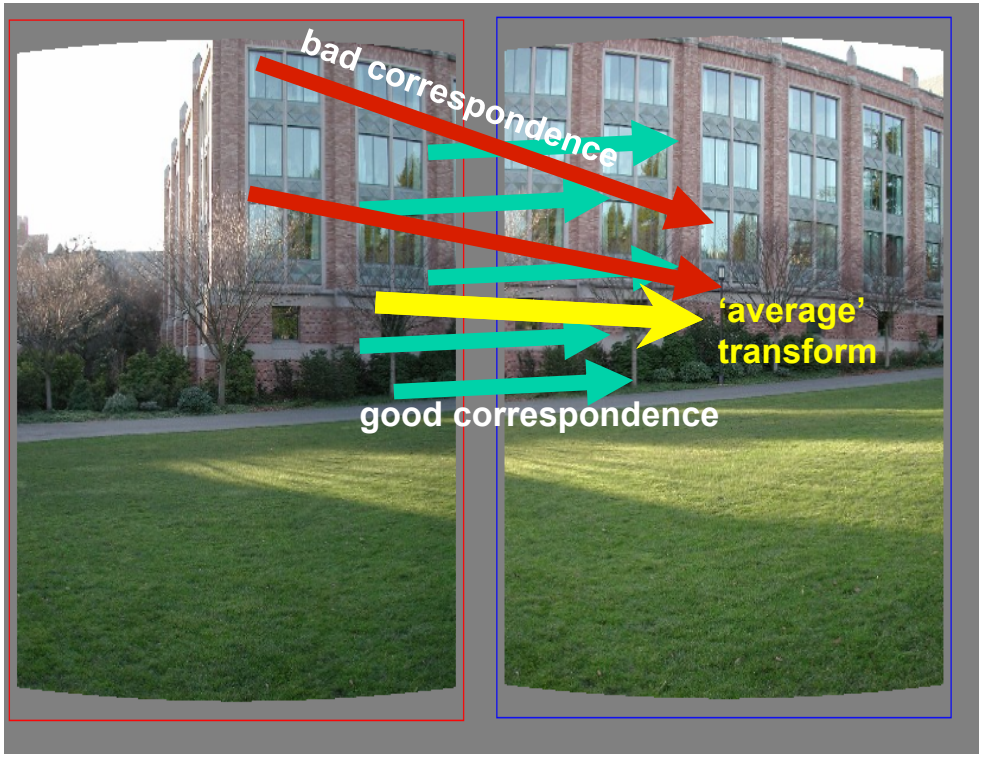
how should we estimate the transform?



Homography

- Estimating homography using RANSAC :
  - LLS will find the ‘average’ transform
  - solution is corrupted

    by bad correspondences



Homography

- Estimating homography using RANSAC :
  - How many correspondences

    to compute translation transform?

Need only **one correspondence**,

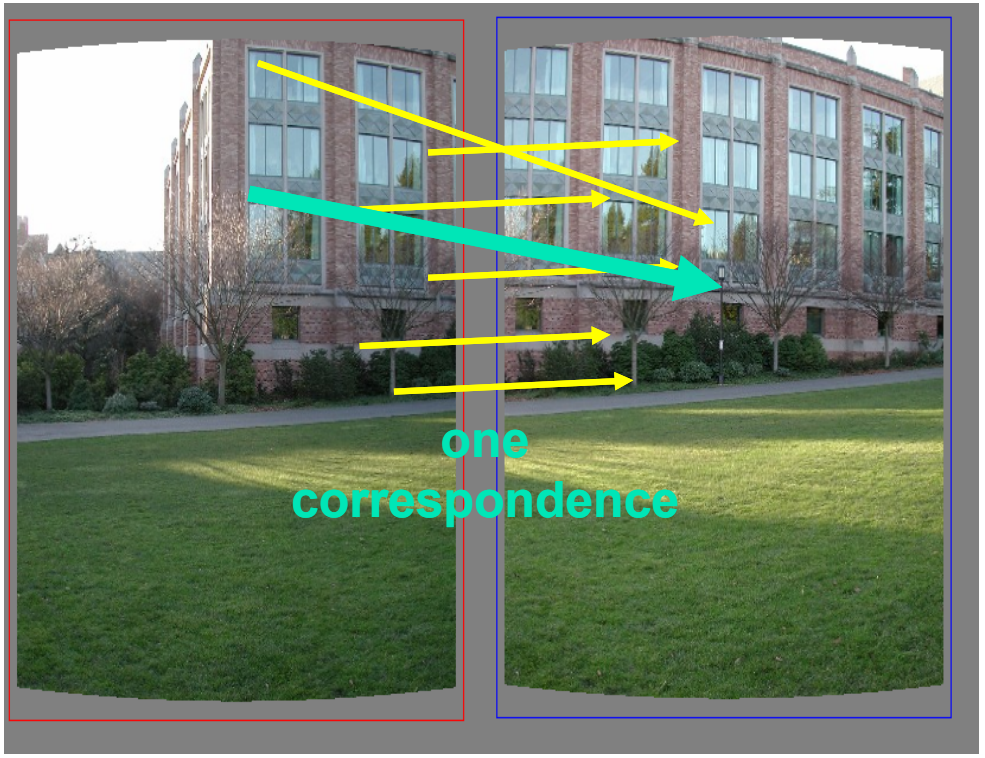
to find translation model



Homography

- Estimating homography using RANSAC :
  - **Pick one correspondence**,

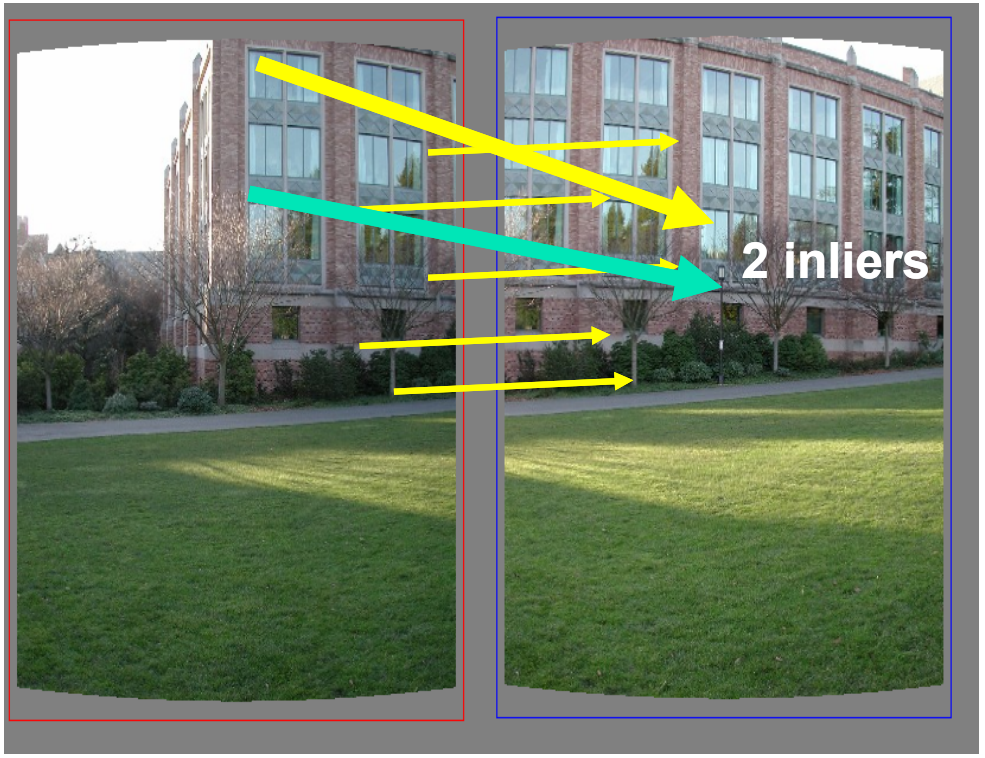
    count inliers



Homography

- Estimating homography using RANSAC :
  - **Pick one correspondence**,

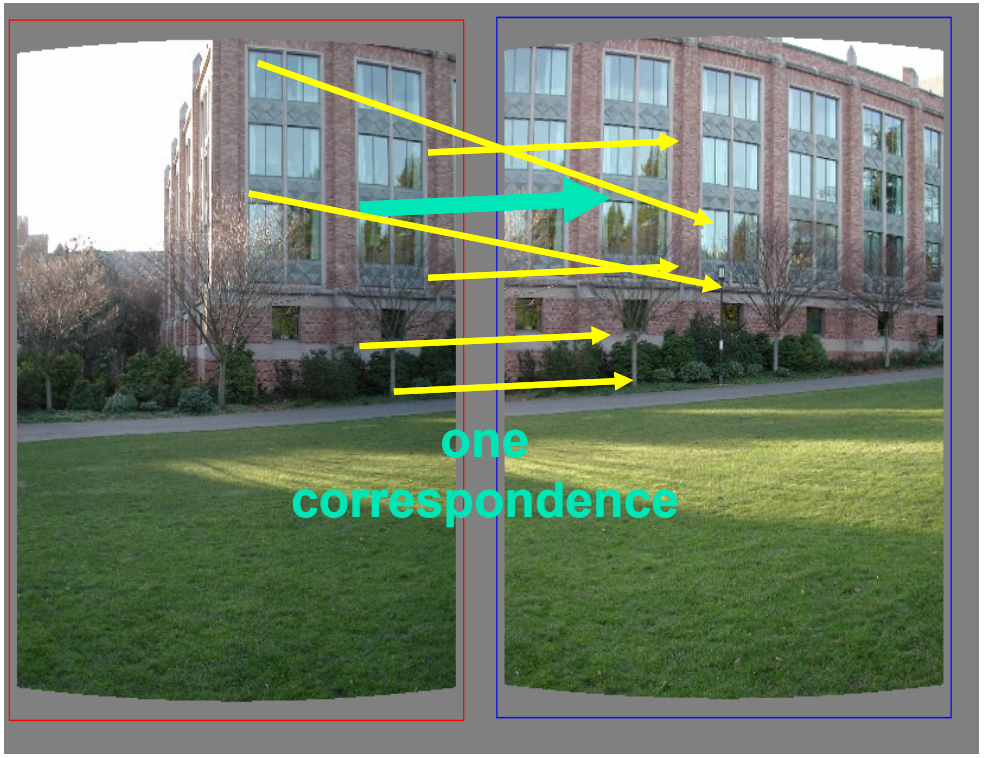
    count inliers



Homography

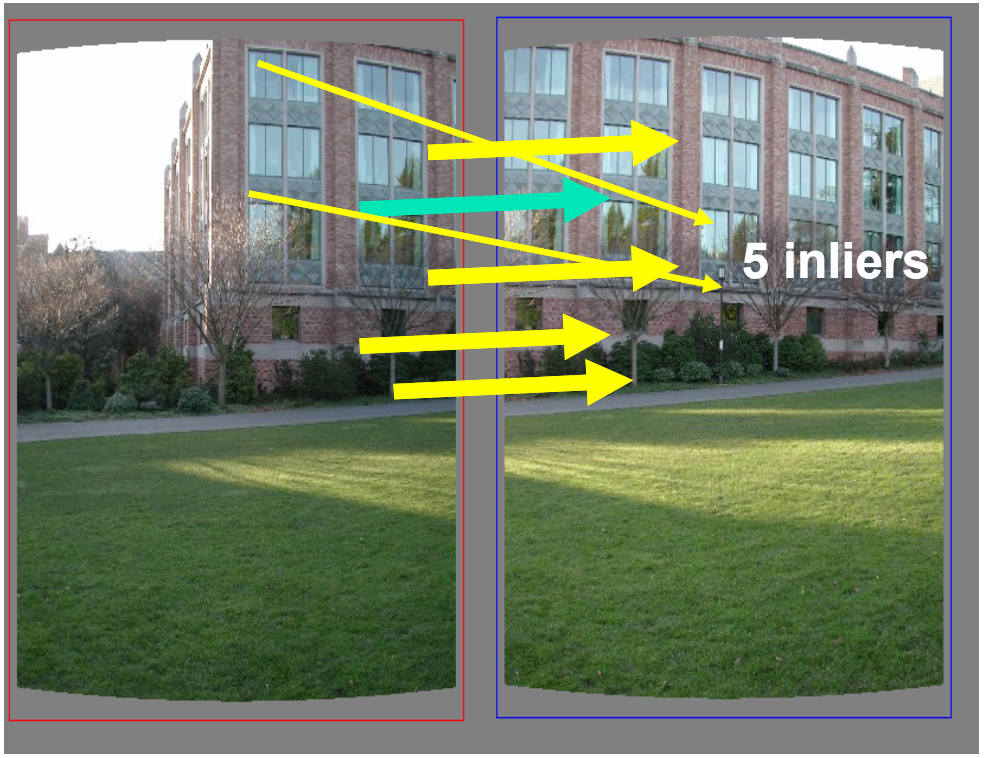
- Estimating homography using RANSAC :
  - **Pick one correspondence**,

    count inliers



Homography

- Estimating homography using RANSAC :
  - **Pick one correspondence**, count inliers
  - Pick the model with the highest number of inliers!



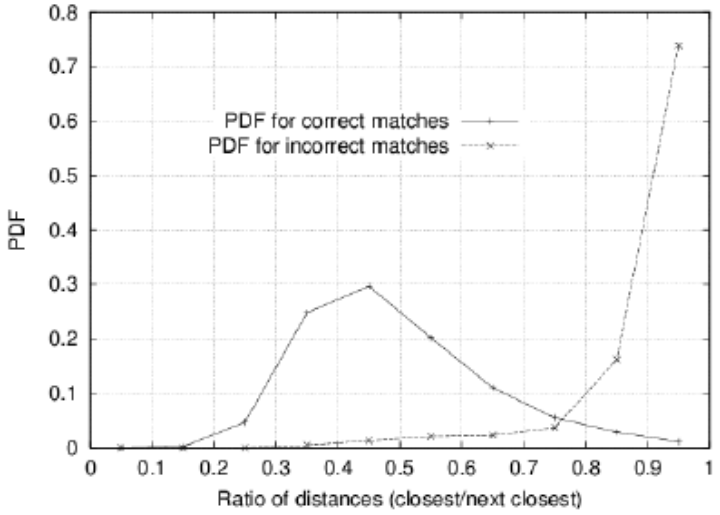
Homography

https://docs.opencv.org/master/d1/de0/tutorial_py_feature_homography.html

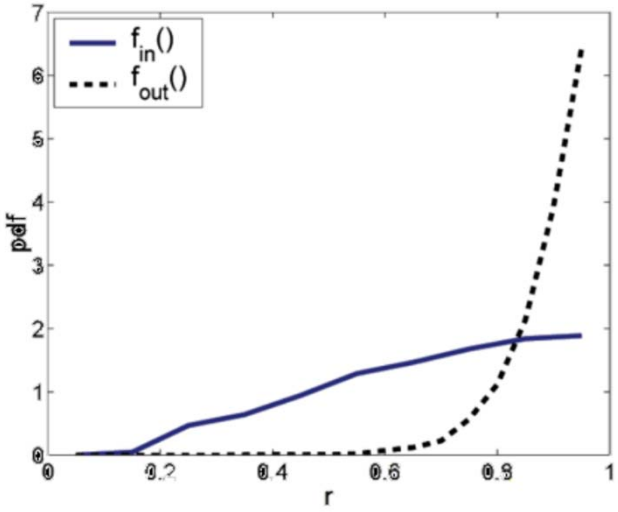
- Estimating homography using RANSAC :
  - RANSAC loop
    1. Get point correspondence (randomly)
    2. Compute H
    3. Count inliers
    4. Keep H if largest number of inliers
  - Recompute H using all inliers

Rejection of Unreliable Matches

- Heuristic: compare distance of **nearest** neighbor to that of **second** nearest neighbor
- Ratio of closest distance to second-closest distance will be *high* for features that are *not* distinctive



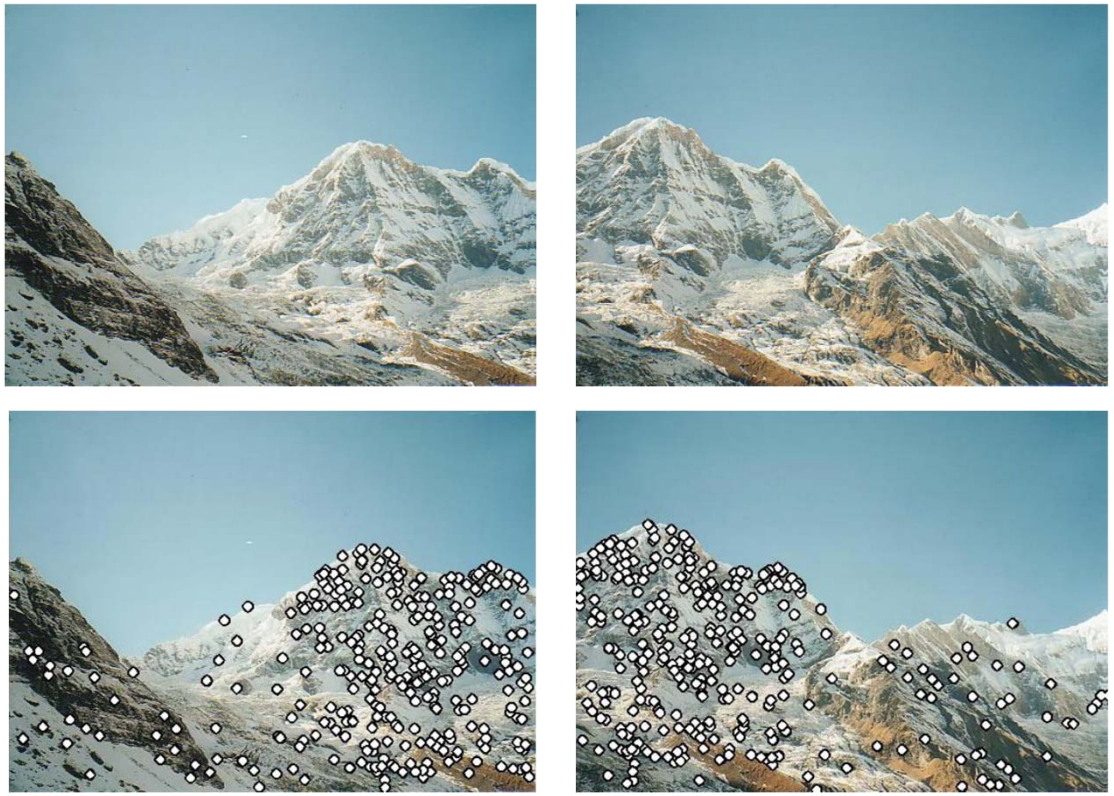
Lowe IJCV04



Goshen&Shimshoni PAMI08

Homography

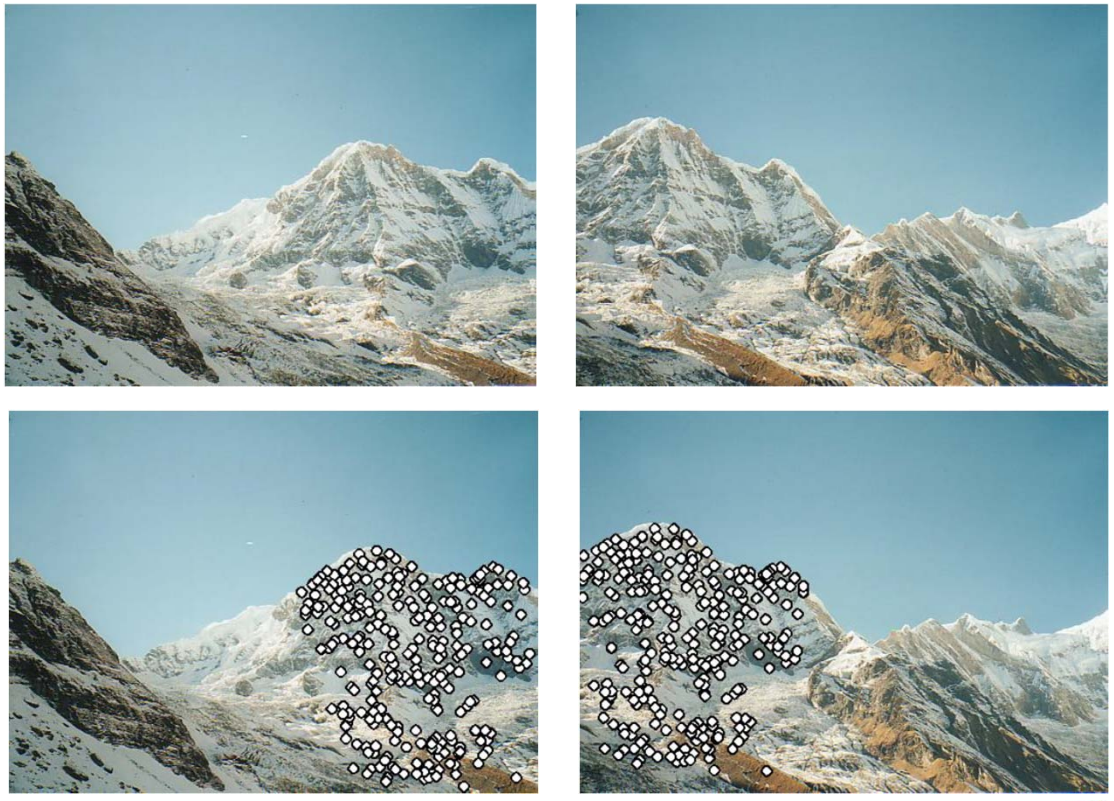
- Useful for… Stitching Images
  - Matching SIFT Features



Homography

- Useful for… Stitching Images
  - Reject Outliers

    using RANSAC



Homography

- Useful for… Stitching Images
  - Stitching Images

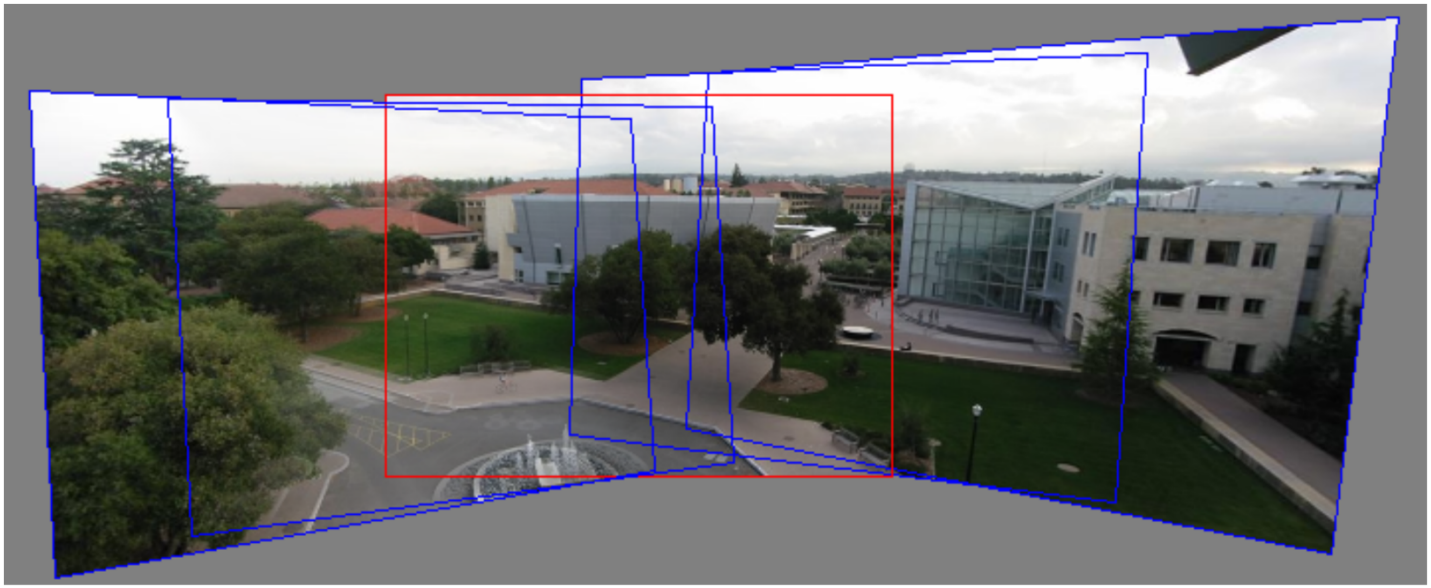






Homography

- Useful for… Panoramas



1. Pick one image (red)
2. Warp the other images towards it (usually, one by one)
3. blend

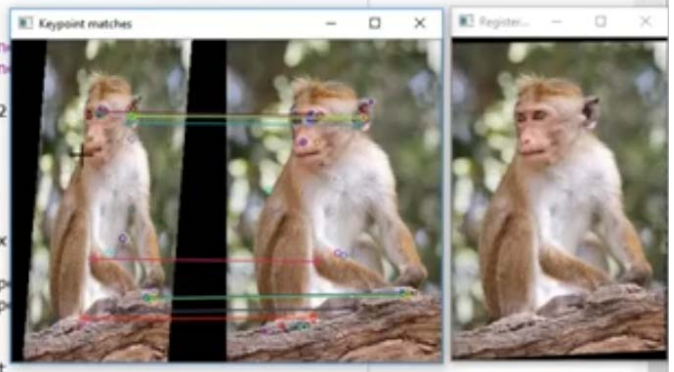
Homography

- Useful for… Panoramas
  1. Take a sequence of images from the same position (rotate the camera about its optical center)
  2. Compute transformation between second image and first
  3. Shift the second image to overlap with the first
  4. Blend the two together to create a mosaic
  5. If there are more images, repeat

Homography

- Image registration by OpenCV with Python

```
19 img1 = cv2.cvtColor(im1, cv2.COLOR_BGR2GRAY)
20 img2 = cv2.cvtColor(im2, cv2.COLOR_BGR2GRAY)
21
22 #Initiate ORB
23 orb = cv2.ORB_create(50)
24
25 kp1, des1 = orb.detectAndCompute(img1, Non
26 kp2, des2 = orb.detectAndCompute(img2, Non
27
28 matcher = cv2.DescriptorMatcher_create(cv2
29
30 #Match descriptors
31
32 matches = matcher.match(des1, des2, None)
33
34 matches = sorted(matches, key = lambda x:x
35
36 points1 = np.zeros((len(matches), 2), dtyp
37 points2 = np.zeros((len(matches), 2), dtyp
38
39 for i, match in enumerate(matches):
40     points1[i, :] = kp1[match.queryIdx].pt
41     points2[i, :] = kp2[match.trainIdx].pt
42
43 h, mask = cv2.findHomography(points1, points2, cv2.RANSAC)
44
45 # USe homography
46
47 height, width, channels = im2.shape
48
49 im1Reg = cv2.warpPerspective(im1, h, (width, height))
50
51 img3 = cv2.drawMatches(im1, kp1, im2, kp2, matches[:10], None)
52 cv2.imshow("Keypoint matches", img3)
53 cv2.imshow("Registered image", im1Reg)
54 cv2.waitKey(0)
```



Registration
- https://youtu.be/cA8K8dl-E6k

Homography

- Image stitching by OpenCV with Python



```
#img_ = cv2.imread('original_image_left.jpg')
#img_ = cv2.resize(img_, (0,0), fx=1, fy=1)
img1 = cv2.cvtColor(img_,cv2.COLOR_BGR2GRAY)

img = cv2.imread('original_image_left.jpg')
#img = cv2.imread('original_image_right.jpg')
#img = cv2.resize(img, (0,0), fx=1, fy=1)
img2 = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

sift = cv2.xfeatures2d.SIFT_create()
# find key points
kp1, des1 = sift.detectAndCompute(img1,None)
kp2, des2 = sift.detectAndCompute(img2,None)

#cv2.imshow('original_image_left_keypoints',cv2.drawKeypoints(img_,kp1,None))

#FLANN_INDEX_KDTREE = 0
#index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
#search_params = dict(checks = 50)
#match = cv2.FlannBasedMatcher(index_params, search_params)
match = cv2.BFMatcher()
matches = match.knnMatch(des1,des2,k=2)

good = []
for m,n in matches:
    if m.distance < 0.03*n.distance:
        good.append(m)

draw_params = dict(matchColor=(0,255,0),
                   singlePointColor=None,
                   flags=2)

img3 = cv2.drawMatches(img_,kp1,img,kp2,good,None,**draw_params)
#cv2.imshow("original_image_drawMatches.jpg", img3)

MIN_MATCH_COUNT = 10
if len(good) > MIN_MATCH_COUNT:
    src_pts = np.float32([ kp1[m.queryIdx].pt for m in good ]).reshape(-1,1,2)
    dst_pts = np.float32([ kp2[m.trainIdx].pt for m in good ]).reshape(-1,1,2)

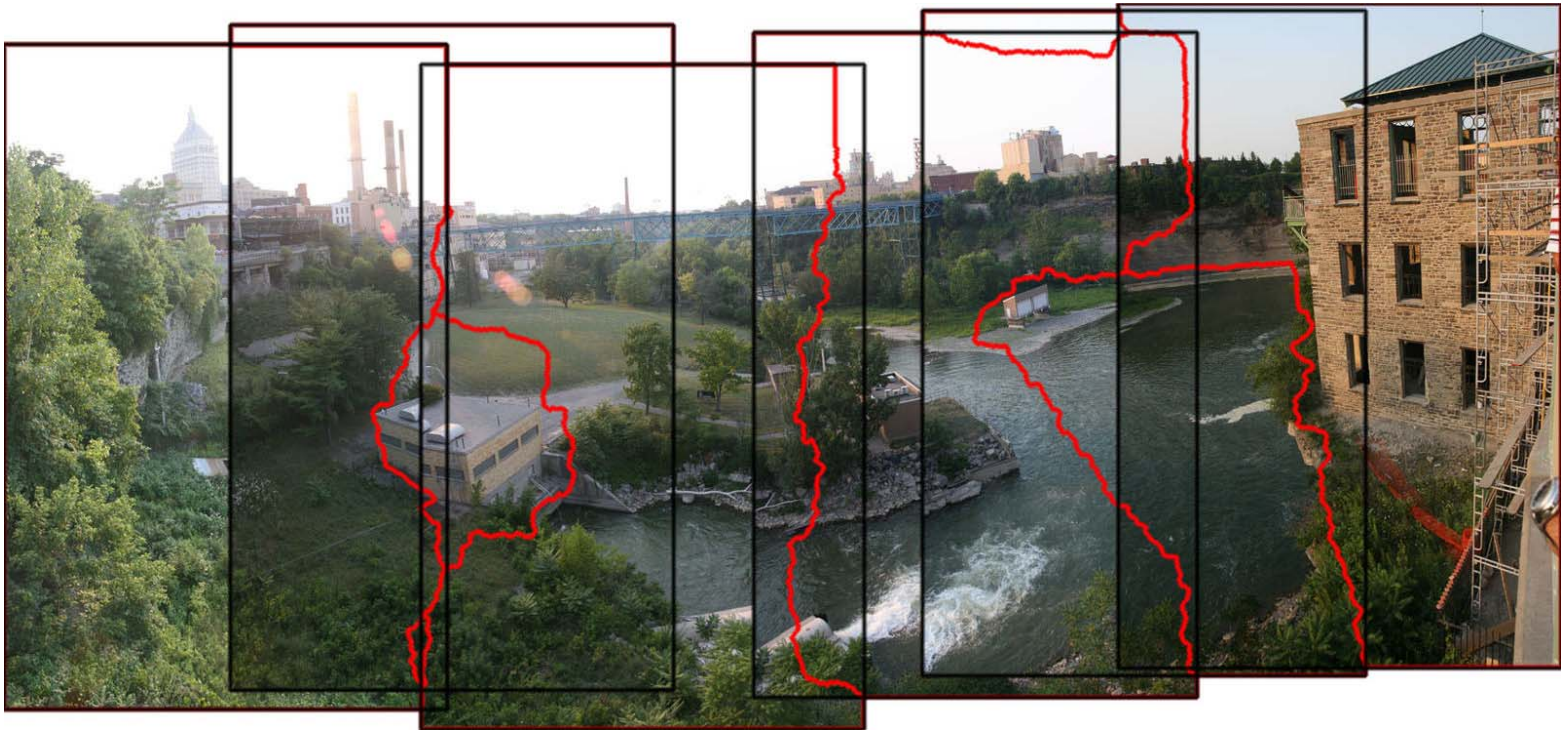
    M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
```

```
RESTART: C:\Users\HOME\Desktop\OpenCV Image Stiching\main.py
```

Image stitching
-(part1) https://youtu.be/ToldvnUtBh0
-(part2) https://youtu.be/mY5jp8KQM6g

Homography

- Useful for… Panoramas



Homography

- Removing Seams :
  - Vignetting – reduction of image brightness at the periphery – caused in digital cameras by angle dependence of the light incidence on the sensor
  - Small misalignments



Homography

- Multi-band Blending :
  - Multi-resolution technique using image pyramid
  - Hides seams but preserves sharp detail



Homography

- Feathering :
  - Encoding transparency

$$I(x,y) = (\alpha R, \alpha G, \alpha B, \alpha)$$

$$I_{blend} = I_{left} + I_{right}$$

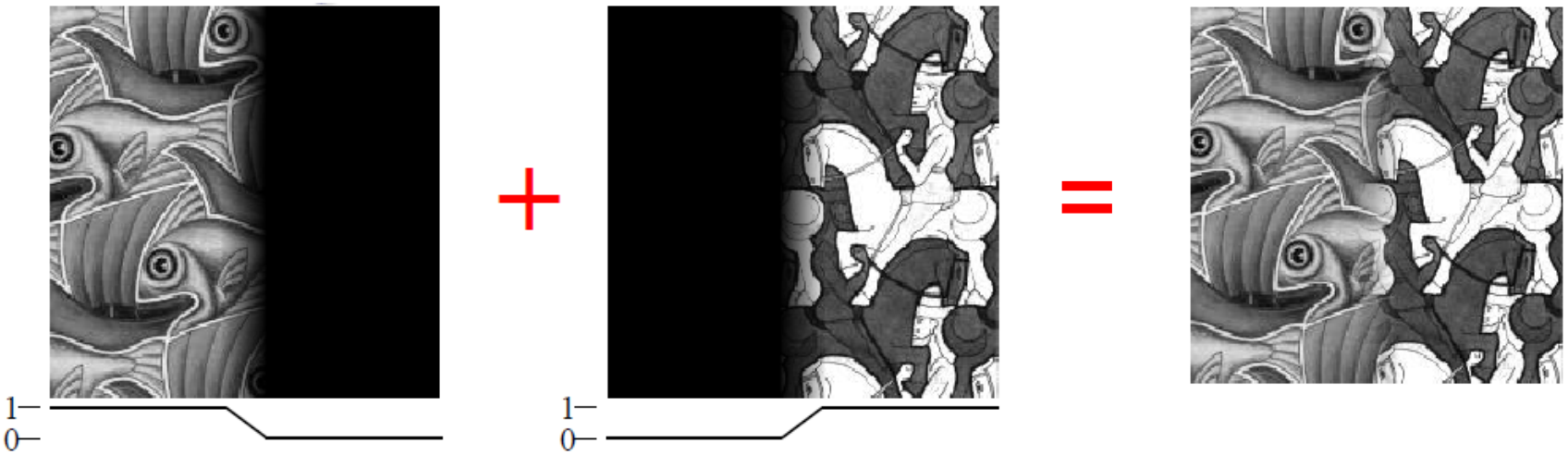
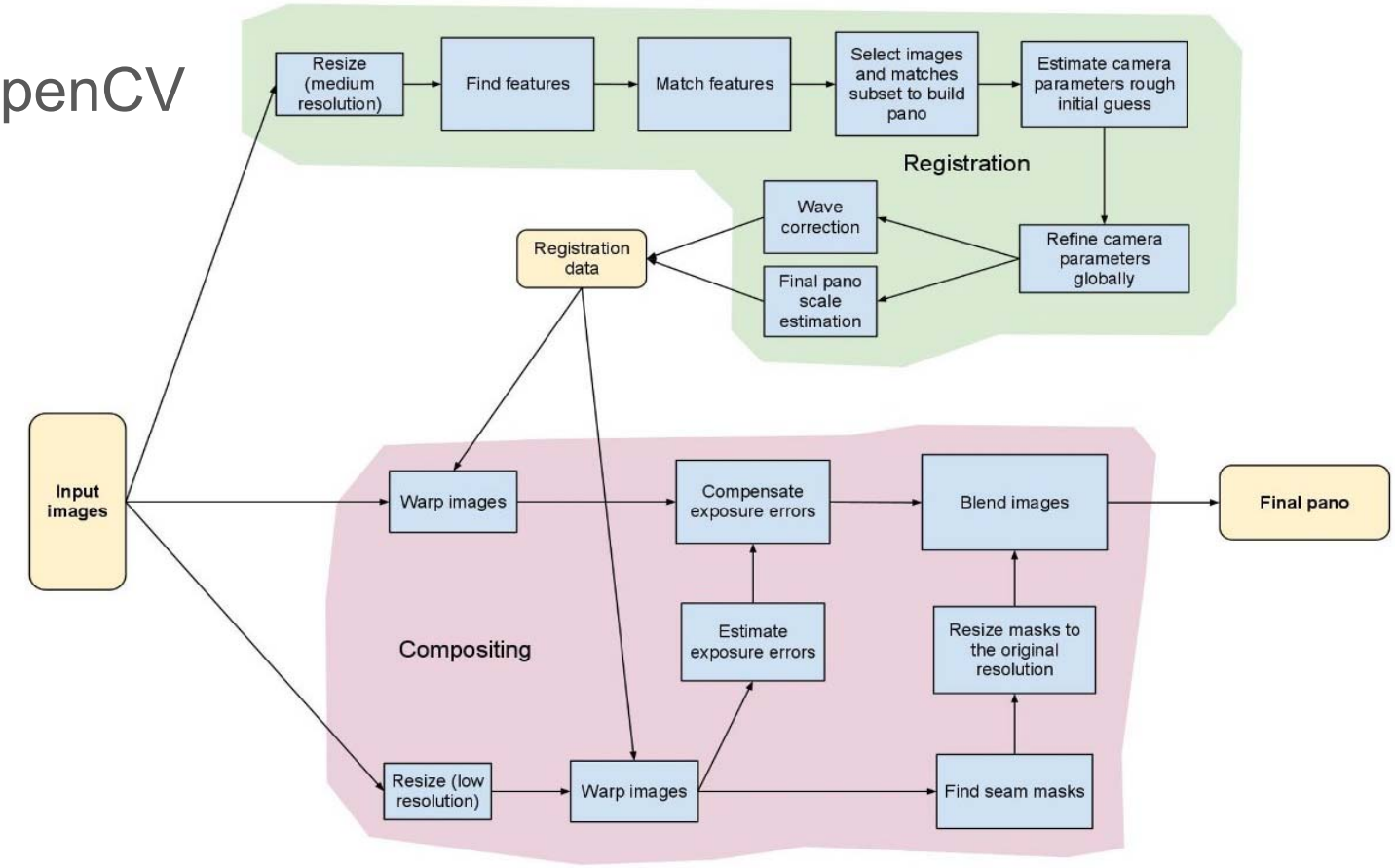


Image mosaic (blending)
http://pages.cs.wisc.edu/~csverma/CS766_09/ImageMosaic/imagemosaic.html

Image Alignment

- Stitching pipeline in openCV



https://www.pyimagesearch.com/2016/01/11/opencv-panorama-stitching/

Thanks!